When personalization deters delegation to algorithms — Replication analysis

Lara Suraci & Irenaeus Wolff

16 Oktober 2026

Contents

1 Data import and preparation **3**
1.1 Shared plotting constants . . . 3
1.2 Load raw data . . . 3
1.3 Preregistered exclusions . . . 3
1.4 Build participant-level Part-1 preference measures . . . 4

2 Descriptive overview **6**
2.1 Choice quality and payoffs . . . 6
2.2 Round-level delegation profile . . . 6
2.3 Figure 1 — Average delegation over decisions . . . 7

3 Primary test 1 — First-contact delegation (round 1) **9**
3.1 Effect size and CI for the round-1 contrast . . . 9

4 Primary test 2 — Delegation across repeated decisions **11**
4.1 Participant-level Wilcoxon–Mann–Whitney test . . . 11
4.2 Effect size and CI for the WMW test . . . 11
4.3 Risk aversion . . . 11
4.4 Figure 3 — ECDF of participant-level average delegation . . . 12
4.5 Preregistered regression specification . . . 13
4.6 Robustness check: logistic regression . . . 14
4.7 Supporting regression . . . 15
4.8 Round-1 outcome and round-2 delegation, within treatment . . . 15
4.9 Exploratory: why does delegation under EV decline after round 1? . . . 18

5 Categorical non-delegation **31**
5.1 Significance test and effect size for categorical non-delegation . . . 32
5.2 Comparisons of never-delegators vs. delegators . . . 33
5.3 Choice quality among never-delegators (Limitations) . . . 37

6 Responses to feedback **40**
6.1 Preregistered secondary hypothesis: switching to non-delegation after a zero payoff . . . 40
6.2 Exploratory: win-stay / lose-shift . . . 42

7 Exploratory mechanisms **45**
7.1 Coefficient model (interaction with performance expectations) . . . 45
7.2 Treatment comparison of expected algorithmic errors . . . 46

8 Post-task reasons **48**

8.1 Figure 4 — Reasons by treatment and delegation type 48
8.2 Aggregate reason patterns 50
8.3 The responsibility items are worded according to the hypothetical choice 51
8.4 Reasons among delegators, by treatment 53
8.5 LPM: never-delegation on reasons 54

9 Exclusion-criterion breakdown (Method section) 57

10 Power analysis (pilot-based, Appendix C of the paper) 59
10.1 Pilot: categorical non-delegation (footnote and Discussion) 59
10.2 Minimum detectable effect (MDE) at 80% power 60
10.3 MDE and power for the average-delegation comparison 60
10.4 Power to re-detect the pilot's observed effect 61

```r
library(dplyr)
library(tidyr)
library(ggplot2)
library(scales)
library(Exact)     # Boschloo exact test
library(fixest)    # LPM with fixed effects + clustered SE
library(lmtest)    # robust coefficient tests
library(sandwich)  # HC2 standard errors
library(texreg)    # regression tables
# library(readODS) # only needed for the Appendix task-matrix chunk near the end
```

1 Data import and preparation

1.1 Shared plotting constants

To keep figures visually consistent we fix the treatment colour palette once.

```
# Treatment colours used throughout (Pref = preference-based, EV = optimisation)
col_pref <- "#4E79A7"
col_ev   <- "#D55E00"
algo_cols <- c("Pref" = col_pref, "EV" = col_ev)
```

1.2 Load raw data

Paths are relative to this script's location (replication-package layout).

```
choices <- read.delim("260206_1528_537_lotteryChoices.xls")
subjects <- read.csv("allSubjects.csv", header = TRUE)
prolificData <- read.csv("prolific_demographic_export_approved.csv")

choicesP1 <- subset(choices, Part == 1)
```

1.3 Preregistered exclusions

We apply the preregistered exclusion rules: failed bot/slider check at first attempt, failed comprehension checks after the second attempt, failure to confirm careful participation (`Reliable < 6`), disclosed AI usage, incomplete data (`Delegated < 0`), and the log-time "too fast" rule.

```
# "Too fast" flag based on the preregistered log-completion-time rule
prolificData$logCompleTime <- log(prolificData$Time.taken)
prolificData$tooFast <- 1 * (
  prolificData$logCompleTime <
    mean(log(prolificData$Time.taken)) - 2 * sd(log(prolificData$Time.taken))
)

keep_ids <- prolificData$Participant.id[prolificData$tooFast == 0]

choices <- subset(
  choices,
  BotCheck1attempt == 1 &
    Part1CQsPassedOnAttempt <= 2 &
    Part2CQsPassedOnAttempt <= 2 &
    Reliable >= 6 &
    AIUsageInStudy == 0 &
    Delegated >= 0 &
    ProlificID %in% keep_ids
)

subjects  <- subset(subjects,  ProlificID %in% choices$ProlificID)
choicesP1 <- subset(choicesP1, ProlificID %in% choices$ProlificID)

# Final analysis sample (paper: N = 233; EV = 118, Pref = 115)
cat("Final sample size per condition (1 = EV, 0 = Preference-based):\n")
```

```
## Final sample size per condition (1 = EV, 0 = Preference-based):
```

```r
table(subjects$EVAlgorithm)
```

```
##
##   0   1
## 115 118
```

```r
# Convenience label for the algorithm condition
choices <- choices %>%
  mutate(Algorithm = ifelse(EVAlgorithm == 1, "EV", "Pref"))
```

1.4 Build participant-level Part-1 preference measures

These variables (revealed risk preferences from Part 1, per-task confidence, and post-task expectations) are merged onto the decision-level `choices` data for use in the regressions and the appendix coefficient plot.

```r
# EV-maximising choice in each Part-1 task
choicesP1 <- choicesP1 %>%
  mutate(EVChoice = case_when(
    grepl("^P", ChoiceType) & PBetChosen == 1 ~ 1,
    grepl("^D", ChoiceType) & PBetChosen == 0 ~ 1,
    TRUE ~ 0
  ))

# "True" P-bet choice (less-risky choice when it was NOT EV-maximising)
choicesP1 <- choicesP1 %>%
  mutate(truePChoice = case_when(
    grepl("^D", ChoiceType) & PBetChosen == 1 ~ 1,
    TRUE ~ 0
  ))

# Participant-level Part-1 summaries
p1_summaries <- choicesP1 %>%
  group_by(ProlificID) %>%
  summarise(
    EVChoices    = sum(EVChoice),
    truePChoices = sum(truePChoice),
    PBet_count   = sum(PBetChosen),
    .groups = "drop"
  )

choices <- choices %>%
  left_join(p1_summaries, by = "ProlificID") %>%
  # per-task Part-1 confidence (matched on participant x option set)
  left_join(
    choicesP1 %>% select(ProlificID, OptionsetID, ConfidenceP1 = Confidence),
    by = c("ProlificID", "OptionsetID")
  ) %>%
  # participant-level demographics and post-task expectations
  left_join(
    subjects %>% select(
      ProlificID, Female, Age, Education, HHIncome,
      ProbOfBadComputerChoices, ProbOfAbsurdComputerChoices, ComputerRiskTaking
    ),
    by = "ProlificID"
```

```
)
```

2 Descriptive overview

2.1 Choice quality and payoffs

Choice quality among non-delegated decisions and the payoff advantage of delegation (paper, Section "Primary test: first-contact delegation").

```
# Share of non-dominated self-chosen options, by condition
choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm) %>%
  summarise(
    avNonDom = mean(Nondominated[Delegated == 0 & Option > 0], na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 2
##   Algorithm avNonDom
##   <chr>        <dbl>
## 1 EV           0.341
## 2 Pref         0.351
```

```
# Average payoff: delegated vs. non-delegated decisions
# (paper: 243.49 points delegated vs. 195.18 points non-delegated, ~25% higher)
choices %>%
  filter(Part == 2) %>%
  group_by(Delegated) %>%
  summarise(meanPayoff = mean(RelevantPayoff, na.rm = TRUE), .groups = "drop")
```

```
## # A tibble: 2 x 2
##   Delegated meanPayoff
##       <int>      <dbl>
## 1         0       195.
## 2         1       243.
```

2.2 Round-level delegation profile

```
choicesAv <- choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm, WithinPartDecisionNumber) %>%
  summarise(
    avDeleg  = mean(Delegated, na.rm = TRUE),
    avNonDom = mean(Nondominated[Delegated == 0 & Option > 0], na.rm = TRUE),
    delegSE = sd(Delegated, na.rm = TRUE) / sqrt(n()),
    .groups = "drop"
  )
choicesAv
```

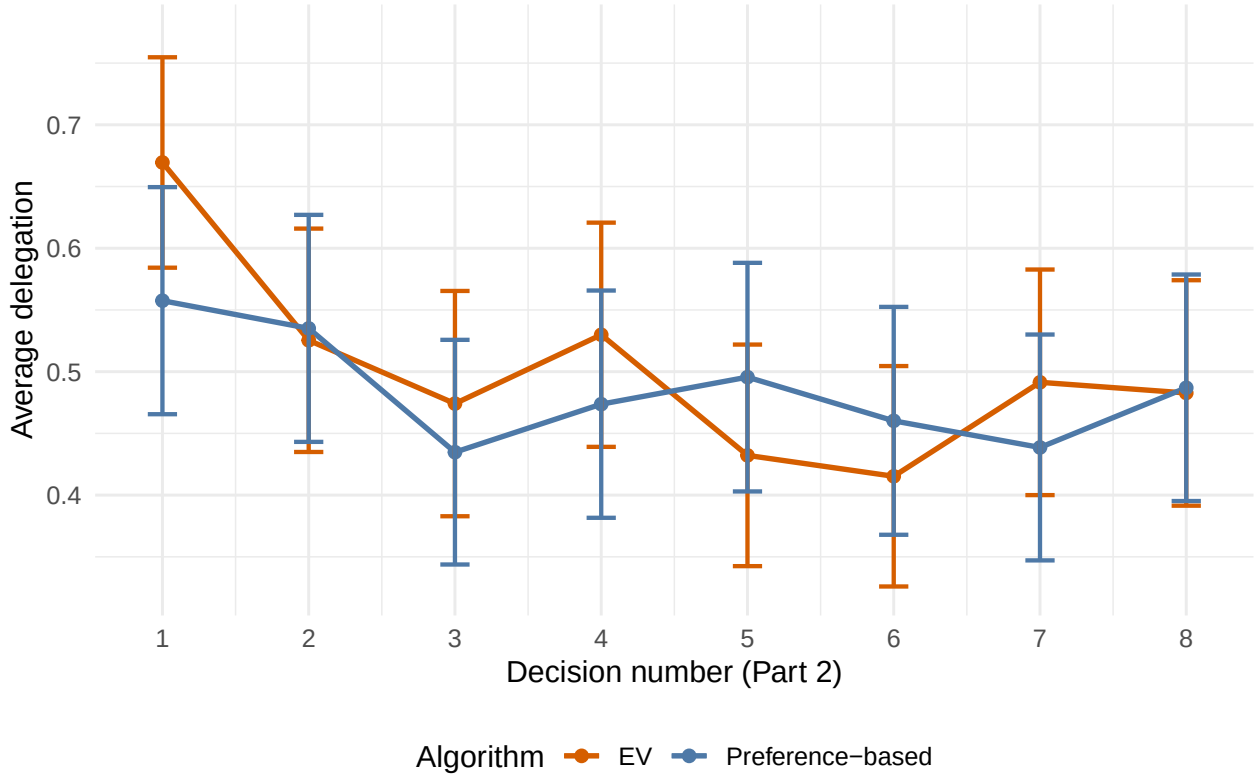
```
## # A tibble: 16 x 5
##    Algorithm WithinPartDecisionNumber avDeleg avNonDom delegSE
##    <chr>                        <int>   <dbl>    <dbl>   <dbl>
##  1 EV                               1   0.669    0.342  0.0435
##  2 EV                               2   0.525    0.214  0.0462
##  3 EV                               3   0.474    0.3    0.0466
##  4 EV                               4   0.530    0.333  0.0463
```

```
##  5 EV                              5   0.432    0.269  0.0458
##  6 EV                              6   0.415    0.464  0.0456
##  7 EV                              7   0.491    0.356  0.0466
##  8 EV                              8   0.483    0.433  0.0466
##  9 Pref                            1   0.558    0.4    0.0469
## 10 Pref                            2   0.535    0.327  0.0469
## 11 Pref                            3   0.435    0.344  0.0464
## 12 Pref                            4   0.474    0.267  0.0470
## 13 Pref                            5   0.496    0.429  0.0472
## 14 Pref                            6   0.460    0.333  0.0471
## 15 Pref                            7   0.439    0.391  0.0467
## 16 Pref                            8   0.487    0.322  0.0468
```

2.3 Figure 1 — Average delegation over decisions

```
fig1 <- choicesAv %>%
  mutate(Algorithm = factor(Algorithm, levels = c("EV", "Pref"),
                            labels = c("EV", "Preference-based"))) %>%
  ggplot(aes(x = WithinPartDecisionNumber, y = avDeleg,
             color = Algorithm, group = Algorithm)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  geom_errorbar(aes(ymin = avDeleg - 1.96 * delegSE, ymax = avDeleg + 1.96 * delegSE),
                width = 0.2, linewidth = 0.8) +
  scale_x_continuous(breaks = 1:8) +
  scale_y_continuous(breaks = (4:7)/10, limits = c(0.325, 0.775)) +
  scale_color_manual(values = c("EV" = col_ev, "Preference-based" = col_pref)) +
  labs(x = "Decision number (Part 2)",
       y = "Average delegation",
       color = "Algorithm") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

fig1
```




```
ggsave("Figure1_AverageDelegation.pdf", fig1, width = 7, height = 4.5)
```

3 Primary test 1 — First-contact delegation (round 1)

Preregistered one-sided Boschloo exact test on round-1 delegation (paper: 66.9% vs. 55.8%, p = 0.041).

```
tab <- table(
  Delegated   = choices$Delegated[choices$Part == 2 & choices$WithinPartDecisionNumber ==
  ↪  1],
  EVAlgorithm = choices$EVAlgorithm[choices$Part == 2 & choices$WithinPartDecisionNumber
  ↪  == 1]
)

# Delegation share by condition (column 2 = EV)
prop.table(tab, margin = 2)
```

```
##          EVAlgorithm
## Delegated         0         1
##         0 0.4424779 0.3305085
##         1 0.5575221 0.6694915
```

```
# One-sided Boschloo exact test (H1: EV > Pref)
exact.test(tab, method = "Boschloo", alternative = "greater", to.plot = FALSE)$p.value
```

```
## [1] 0.0412565
```

3.1 Effect size and CI for the round-1 contrast

Risk difference (Wald 95% CI) and odds ratio (Fisher's exact two-sided 95% CI), as reported in the manuscript's new Table 1 (RD = 11.1pp [-1.3, 23.7]; OR = 1.60 [0.91, 2.84]).

```
p_ev   <- prop.table(tab, margin = 2)["1", "1"]
p_pref <- prop.table(tab, margin = 2)["1", "0"]
n_ev   <- sum(tab[, "1"])
n_pref <- sum(tab[, "0"])

# Risk difference + Wald 95% CI
rd <- p_ev - p_pref
se_rd <- sqrt(p_ev * (1 - p_ev) / n_ev + p_pref * (1 - p_pref) / n_pref)
cat("Risk difference:", round(rd, 3), "\n")
```

```
## Risk difference: 0.112
```

```
cat("Wald 95% CI:", round(rd - 1.96 * se_rd, 3), "to", round(rd + 1.96 * se_rd, 3), "\n")
```

```
## Wald 95% CI: -0.013 to 0.237
```

```
# Odds ratio + Fisher's exact two-sided 95% CI
fisher.test(tab)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  tab
## p-value = 0.1044
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.9109897 2.8414026
## sample estimates:
```

```
## odds ratio
##   1.604278
```

4 Primary test 2 — Delegation across repeated decisions

4.1 Participant-level Wilcoxon–Mann–Whitney test

Preregistered one-sided WMW test on participant-average delegation (paper: W = 6428, p = 0.243).

```
delegAv <- choices %>%
  filter(Part == 2) %>%
  group_by(Algorithm, ProlificID) %>%
  summarise(avDeleg = mean(Delegated, na.rm = TRUE), .groups = "drop") %>%
  mutate(Algorithm = factor(Algorithm, levels = c("Pref", "EV")))

wilcox.test(avDeleg ~ Algorithm, data = delegAv, alternative = "less")
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  avDeleg by Algorithm
## W = 6428, p-value = 0.2425
## alternative hypothesis: true location shift is less than 0
```

4.2 Effect size and CI for the WMW test

Rank-biserial correlation with a bootstrap 95% CI, as reported in the manuscript's new Table 1 ($r_{rb} \approx 0.05$, bootstrap CI [-0.10, 0.20]).

```
rank_biserial <- function(x, y) {
  n1 <- length(x); n2 <- length(y)
  U <- wilcox.test(x, y)$statistic
  1 - (2 * U) / (n1 * n2)
}

x_pref <- delegAv$avDeleg[delegAv$Algorithm == "Pref"]
x_ev   <- delegAv$avDeleg[delegAv$Algorithm == "EV"]

r_rb <- rank_biserial(x_pref, x_ev)
cat("Rank-biserial r:", round(r_rb, 3), "\n")
```

```
## Rank-biserial r: 0.053
```

```
set.seed(1)
B <- 2000
boot_r <- replicate(B, {
  xb <- sample(x_pref, replace = TRUE)
  yb <- sample(x_ev,   replace = TRUE)
  rank_biserial(xb, yb)
})
cat("Bootstrap 95% CI:", round(quantile(boot_r, 0.025), 3), "to",
    round(quantile(boot_r, 0.975), 3), "\n")
```

```
## Bootstrap 95% CI: -0.101 to 0.198
```

4.3 Risk aversion

```
# calculate how often the EV-algorithm chooses p-bets
table(choicesP1$ChoiceType[choicesP1$OptionsetID <= 8] == "P105")
```

```
##
## FALSE  TRUE
##   932   932
```

```
# 50-50; contrast that to...
pBetChoices <- choicesP1 %>%
  filter(OptionsetID <= 8) %>% # among the relevant Option sets
  group_by(ProlificID) %>%
  summarise(mean(PBetChosen)) # how often did the participant choose a p-bet
prop.table(table(pBetChoices > 0.5))
```

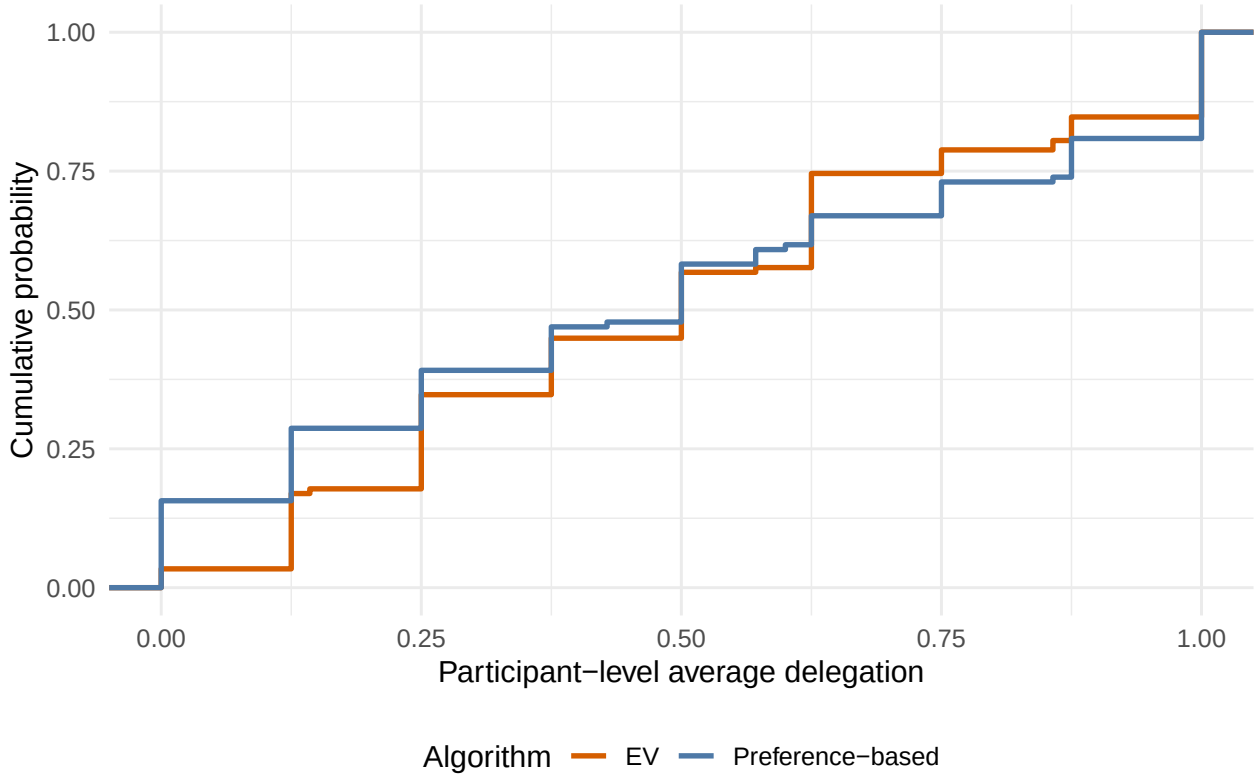
```
##
##     FALSE      TRUE
## 0.1030043 0.8969957
```

for 89.7% of the participants, the prefs-algorithm in expectation will produce fewer zero-outcomes than the EV-algorithm

4.4 Figure 3 — ECDF of participant-level average delegation

```
fig3 <- delegAv %>%
  mutate(Algorithm = factor(Algorithm, levels = c("EV", "Pref"),
                            labels = c("EV", "Preference-based"))) %>%
  ggplot(aes(x = avDeleg, color = Algorithm)) +
  stat_ecdf(linewidth = 1) +
  scale_color_manual(values = c("EV" = col_ev, "Preference-based" = col_pref)) +
  labs(x = "Participant-level average delegation",
       y = "Cumulative probability",
       color = "Algorithm") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

fig3
```




```
ggsave("Figure3_ECDF.pdf", fig3, width = 7, height = 4.5)
```

4.5 Preregistered regression specification

LPM with decision-set fixed effects and participant-clustered SEs. We first construct the feedback-history regressor (running share of zero outcomes among prior delegated choices).

```
choices <- choices %>%
  arrange(ProlificID, WithinPartDecisionNumber) %>%
  group_by(ProlificID) %>%
  mutate(
    delegated_round = (Delegated == 1),
    zero_payoff     = (RelevantPayoff == 0 & Delegated == 1),
    cum_delegated   = lag(cumsum(delegated_round), default = 0),
    cum_zero        = lag(cumsum(zero_payoff), default = 0),
    zero_rate       = ifelse(cum_delegated > 0, cum_zero / cum_delegated, NA)
  ) %>%
  ungroup()
```

```
# Preregistered model: treatment x (period + feedback history)
delRegPreReg <- feols(
  Delegated ~ EVAlgorithm * (WithinPartDecisionNumber + zero_rate) | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegPreReg)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,324
```

```
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                    Estimate Std. Error   t value   Pr(>|t|)
## EVAlgorithm                       -0.048717   0.083122 -0.586090 5.5845e-01
## WithinPartDecisionNumber          -0.001795   0.008257 -0.217380 8.2813e-01
## zero_rate                         -0.385350   0.078139 -4.931567 1.6664e-06
## EVAlgorithm:WithinPartDecisionNumber  0.001877   0.011504  0.163119 8.7058e-01
## EVAlgorithm:zero_rate             -0.049677   0.097673 -0.508609 6.1157e-01
##
## EVAlgorithm
## WithinPartDecisionNumber
## zero_rate                         ***
## EVAlgorithm:WithinPartDecisionNumber
## EVAlgorithm:zero_rate
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.467127     Adj. R2: 0.115951
##                  Within R2: 0.107265
```

4.6 Robustness check: logistic regression

Re-estimating the preregistered LPM as a logistic regression with the same fixed effects and cluster-robust SEs (paper: EVAlgorithm OR $= 0.79$, $p = 0.549$; zero-outcome-history OR $= 0.19$, $p < 0.001$).

```
delRegLogit <- feglm(
  Delegated ~ EVAlgorithm * (WithinPartDecisionNumber + zero_rate) | OptionsetID,
  family = "logit",
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegLogit)
```

```
## GLM estimation, family = binomial(link = "logit"), Dep. Var.: Delegated
## Observations: 1,324
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                    Estimate Std. Error   z value   Pr(>|z|)
## EVAlgorithm                       -0.240874   0.402319 -0.598714 5.4936e-01
## WithinPartDecisionNumber          -0.010738   0.037790 -0.284144 7.7630e-01
## zero_rate                         -1.661216   0.371278 -4.474320 7.6655e-06
## EVAlgorithm:WithinPartDecisionNumber  0.013504   0.053109  0.254272 7.9929e-01
## EVAlgorithm:zero_rate             -0.241165   0.474017 -0.508770 6.1091e-01
##
## EVAlgorithm
## WithinPartDecisionNumber
## zero_rate                         ***
## EVAlgorithm:WithinPartDecisionNumber
## EVAlgorithm:zero_rate
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## Log-Likelihood:  -829.7   Adj. Pseudo R2: 0.080391
##            BIC: 1,752.9     Squared Cor.: 0.125443
```

```
# Odds ratios
exp(coef(delRegLogit))
```

```
##                           EVAlgorithm             WithinPartDecisionNumber
##                             0.7859404                            0.9893196
##                             zero_rate EVAlgorithm:WithinPartDecisionNumber
##                             0.1899080                            1.0135957
##               EVAlgorithm:zero_rate
##                             0.7857117
```

4.7 Supporting regression

Plain treatment effect (all rounds; excluding round 1; round 1 only). The round-1-only LPM reproduces the Boschloo result ($\beta = 0.113$, one-sided $p = 0.041$).

```
# 1st-round regression in the spirit of the preregistered model: treatment (fixing period
↪  to 1, and taking out the [non-existing] feedback history)
delRegPreReg <- feols(
  Delegated ~ EVAlgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber == 1)
)
summary(delRegPreReg) # Note: two-sided p-value
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 231
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##             Estimate Std. Error t value Pr(>|t|)
## EVAlgorithm 0.112868     0.0648 1.74178 0.082884 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.474644     Adj. R2: 0.0145
##                  Within R2: 0.013716
```

4.8 Round-1 outcome and round-2 delegation, within treatment

The regression above is correlational across the whole sample. To probe the feedback-driven account with variation that is not confounded by the between-treatment difference in decision rule, we condition on having delegated in round 1 and compare round-2 delegation after a zero vs. a positive round-1 payoff, separately by treatment (the payoff depends on which lottery state was drawn). Paper: EV 36.0% vs. 64.8% (two-sided Boschloo $p = 0.020$; OR $= 0.31$, 95% CI [0.10, 0.91] from Fisher's exact test); Pref 50.0% vs. 72.5% (Boschloo $p = 0.078$; OR $= 0.39$, 95% CI [0.11, 1.29]).

```
round12 <- choices %>%
  filter(Part == 2, WithinPartDecisionNumber %in% 1:2) %>%
  select(ProlificID, EVAlgorithm, WithinPartDecisionNumber, Delegated, RelevantPayoff)
  ↪  %>%
  pivot_wider(names_from = WithinPartDecisionNumber,
              values_from = c(Delegated, RelevantPayoff)) %>%
  filter(Delegated_1 == 1, !is.na(Delegated_2)) %>%
  mutate(ZeroPayoff1 = factor(1 * (RelevantPayoff_1 == 0), levels = c(1, 0),
                              labels = c("zero", "positive")),
         Delegated2  = factor(Delegated_2, levels = c(1, 0),
                              labels = c("delegated", "not")))

for (trmt in c(1, 0)) {
  cat("\n====", ifelse(trmt == 1, "EV", "Preference-based"), "====\n")
```

```r
  d <- filter(round12, EVAlgorithm == trmt)
  tab <- table(ZeroPayoff1 = d$ZeroPayoff1, Delegated2 = d$Delegated2)
  print(tab)
  print(round(prop.table(tab, margin = 1), 3))    # share delegating again
  # Paper: two-sided Boschloo p-value; Fisher's exact test supplies the odds ratio and CI
  cat("Two-sided Boschloo p =",
      round(exact.test(tab, method = "Boschloo", to.plot = FALSE)$p.value, 3), "\n")
  print(fisher.test(tab))
}
```

```
##
## ==== EV ====
##            Delegated2
## ZeroPayoff1 delegated not
##    zero              9  16
##    positive         35  19
##            Delegated2
## ZeroPayoff1 delegated   not
##    zero         0.360 0.640
##    positive     0.648 0.352
## Two-sided Boschloo p = 0.02
##
##  Fisher's Exact Test for Count Data
##
## data:  tab
## p-value = 0.02749
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.09974015 0.91067342
## sample estimates:
## odds ratio
##  0.3102391
##
##
## ==== Preference-based ====
##            Delegated2
## ZeroPayoff1 delegated not
##    zero             11  11
##    positive         29  11
##            Delegated2
## ZeroPayoff1 delegated   not
##    zero         0.500 0.500
##    positive     0.725 0.275
## Two-sided Boschloo p = 0.078
##
##  Fisher's Exact Test for Count Data
##
## data:  tab
## p-value = 0.09936
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##  0.1118206 1.2898267
## sample estimates:
```

```
## odds ratio
##     0.3857
```

Do the two within-treatment contrasts differ from each other? (Paper: treatment-by-payoff interaction in a logistic regression, $p = 0.772$.) Also, the share of round-1 delegators who received a zero payoff is not lower under PREF (EV $25/79 = 31.6\%$; PREF $22/62 = 35.5\%$ among those with valid round-2 data); the weaker significance under PREF reflects the smaller number of round-1 delegators (62 vs. 79), not fewer zero outcomes.

```
round12_d <- round12 %>%
  mutate(Zero = 1 * (ZeroPayoff1 == "zero"),
         Del2 = 1 * (Delegated2 == "delegated"))
summary(glm(Del2 ~ EVAlgorithm * Zero, family = binomial, data = round12_d))$coefficients
```

```
##                     Estimate Std. Error    z value    Pr(>|z|)
## (Intercept)        0.9694006  0.3541069  2.7375933 0.006189056
## EVAlgorithm       -0.3584915  0.4545268 -0.7887135 0.430279458
## Zero              -0.9694006  0.5542651 -1.7489835 0.080293880
## EVAlgorithm:Zero -0.2168727  0.7496826 -0.2892860 0.772362521
```

```
round12_d %>%
  group_by(EVAlgorithm) %>%
  summarise(n = n(), zeroShare = mean(Zero))
```

```
## # A tibble: 2 x 3
##   EVAlgorithm     n zeroShare
##         <int> <int>     <dbl>
## 1           0    62     0.355
## 2           1    79     0.316
```

what happens after non-delegation?

```
round12non <- choices %>%
  filter(Part == 2, WithinPartDecisionNumber %in% 1:2) %>%
  select(ProlificID, EVAlgorithm, WithinPartDecisionNumber, Delegated, RelevantPayoff)
  ↪  %>%
  pivot_wider(names_from = WithinPartDecisionNumber,
              values_from = c(Delegated, RelevantPayoff)) %>%
  filter(Delegated_1 == 0, !is.na(Delegated_2)) %>%
  mutate(ZeroPayoff1 = factor(1 * (RelevantPayoff_1 == 0), levels = c(1, 0),
                              labels = c("zero", "positive")),
         Delegated2  = factor(Delegated_2, levels = c(1, 0),
                              labels = c("delegated", "not")))

for (trmt in c(1, 0)) {
  cat("\n====", ifelse(trmt == 1, "EV", "Preference-based"), "====\n")
  nond <- filter(round12non, EVAlgorithm == trmt)
  nontab <- table(ZeroPayoff1 = nond$ZeroPayoff1, Delegated2 = nond$Delegated2)
  print(nontab)
  print(round(prop.table(nontab, margin = 1), 3))   # share delegating again
  cat("Two-sided Boschloo p =",
      round(exact.test(nontab, method = "Boschloo", to.plot = FALSE)$p.value, 3), "\n")
}
```

```
##
## ==== EV ====
```

```
##            Delegated2
## ZeroPayoff1 delegated not
##    zero             5   8
##    positive        13  13
##            Delegated2
## ZeroPayoff1 delegated   not
##    zero         0.385 0.615
##    positive     0.500 0.500
## Two-sided Boschloo p = 0.695
##
## ==== Preference-based ====
##            Delegated2
## ZeroPayoff1 delegated not
##    zero             5   9
##    positive        16  20
##            Delegated2
## ZeroPayoff1 delegated   not
##    zero         0.357 0.643
##    positive     0.444 0.556
## Two-sided Boschloo p = 0.72
```

4.9 Exploratory: why does delegation under EV decline after round 1?

The following chunks are exploratory and were not preregistered. They probe three candidate explanations for the larger round-1-to-round-2 decline in delegation under the EV algorithm: (i) differential exposure to zero payoffs, (ii) disappointment when the EV pick differs from the participant's own Part-1 preference, and (iii) a first-contact difference that is carried by a subgroup of never-delegators. Participant-clustered standard errors are used throughout.

4.9.1 (i) Exposure to zero payoffs

For each delegated decision we compute the probability that the algorithm's pick pays zero (share of the ten equally likely states in which the option pays zero) and compare it with the realized zero share. Paper: expected 41.7% (EV) vs. 34.8% (Pref); realized 40.3% vs. 36.2%, a difference of 4.1 percentage points (clustered $p = 0.18$).

```
# Payoff matrix of the Part-2 option sets, read from the raw decision-level file
raw_choices <- read.delim("260206_1528_537_lotteryChoices.xls")
state_cols  <- grep("^State", names(raw_choices), value = TRUE)

payoff_matrix <- raw_choices %>%
  filter(Part == 2, Option > 0, OptionsetID > 0) %>%
  select(OptionsetID, Option, all_of(state_cols)) %>%
  distinct(OptionsetID, Option, .keep_all = TRUE)
payoff_matrix$pZero <- rowMeans(payoff_matrix[, state_cols] == 0)

delegated_rounds <- choices %>%
  filter(Part == 2, Delegated == 1, Option > 0) %>%
  left_join(payoff_matrix %>% select(OptionsetID, Option, pZero),
            by = c("OptionsetID", "Option")) %>%
  mutate(Zero = 1 * (RelevantPayoff == 0))

delegated_rounds %>%
  group_by(Algorithm) %>%
  summarise(n = n(),
```

```
            participants = n_distinct(ProlificID),
            expectedZero = round(mean(pZero, na.rm = TRUE), 3),
            realizedZero = round(mean(Zero), 3),
            .groups = "drop")
```

```
## # A tibble: 2 x 5
##   Algorithm     n participants expectedZero realizedZero
##   <chr>     <int>        <int>        <dbl>        <dbl>
## 1 EV          471          114        0.417        0.403
## 2 Pref        442           97        0.348        0.362
```

```
m_zero <- glm(Zero ~ EVAlgorithm, family = binomial, data = delegated_rounds)
coeftest(m_zero, vcov = vcovCL(m_zero, cluster = delegated_rounds$ProlificID))
```

```
##
## z test of coefficients:
##
##              Estimate Std. Error z value  Pr(>|z|)
## (Intercept) -0.566733   0.091992 -6.1607 7.243e-10 ***
## EVAlgorithm  0.175403   0.130316  1.3460    0.1783
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.9.2 (ii) Does the EV pick match the participant's Part-1 preference?

Under the preference-based algorithm the delegated option is by construction the participant's preferred option. Under EV it equals that option in about two thirds of delegated rounds. We split EV delegations into "match" and "mismatch" and compare next-round delegation with the preference-based group, with and without controlling for a zero payoff. Paper (all rounds 1–7, 799 delegated rounds): zero share 36.0% (Pref), 36.3% (EV match), 50.8% (EV mismatch); next-round delegation 69.9%, 61.2%, 55.3%.

```
delegation_panel <- choices %>%
  filter(Part == 2) %>%
  arrange(ProlificID, WithinPartDecisionNumber) %>%
  group_by(ProlificID) %>%
  mutate(NextDelegated = lead(Delegated)) %>%
  ungroup() %>%
  filter(Delegated == 1, !is.na(NextDelegated), NextDelegated >= 0) %>%
  mutate(Zero = 1 * (RelevantPayoff == 0),
         Group = factor(ifelse(EVAlgorithm == 0, "Pref",
                         ifelse(Option == PrefAlgoChoice, "EV_match", "EV_mismatch")),
                         levels = c("Pref", "EV_match", "EV_mismatch")))

# Logit with participant-clustered standard errors
cl_test <- function(formula, data) {
  m <- glm(formula, family = binomial, data = data)
  round(coeftest(m, vcov = vcovCL(m, cluster = data$ProlificID)), 3)
}

delegation_panel %>%
  group_by(Group) %>%
  summarise(n = n(), zeroShare = round(mean(Zero), 3),
            nextDelegation = round(mean(NextDelegated), 3), .groups = "drop")
```

```
## # A tibble: 3 x 4
```

```
##   Group           n zeroShare nextDelegation
##   <fct>       <int>     <dbl>          <dbl>
## 1 Pref          386     0.36           0.699
## 2 EV_match      281     0.363          0.612
## 3 EV_mismatch   132     0.508          0.553
```

```
delegation_panel %>%
  group_by(Group, Zero) %>%
  summarise(n = n(), nextDelegation = round(mean(NextDelegated), 3), .groups = "drop")
```

```
## # A tibble: 6 x 4
##   Group        Zero     n nextDelegation
##   <fct>       <dbl> <int>          <dbl>
## 1 Pref            0   247          0.785
## 2 Pref            1   139          0.547
## 3 EV_match        0   179          0.682
## 4 EV_match        1   102          0.49
## 5 EV_mismatch     0    65          0.738
## 6 EV_mismatch     1    67          0.373
```

```
cat("\n--- All rounds 1-7: groups only\n")
```

```
##
## --- All rounds 1-7: groups only
```

```
cl_test(NextDelegated ~ Group + WithinPartDecisionNumber, delegation_panel)
```

```
##
## z test of coefficients:
##
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  0.417      0.204   2.041    0.041 *
## GroupEV_match               -0.378      0.237  -1.594    0.111
## GroupEV_mismatch            -0.632      0.274  -2.309    0.021 *
## WithinPartDecisionNumber     0.113      0.033   3.408    0.001 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat("\n--- All rounds 1-7: groups + zero payoff\n")
```

```
##
## --- All rounds 1-7: groups + zero payoff
```

```
cl_test(NextDelegated ~ Group + Zero + WithinPartDecisionNumber, delegation_panel)
```

```
##
## z test of coefficients:
##
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  0.791      0.210   3.761   <2e-16 ***
## GroupEV_match               -0.401      0.248  -1.620    0.105
## GroupEV_mismatch            -0.503      0.280  -1.798    0.072 .
## Zero                        -1.121      0.152  -7.363   <2e-16 ***
## WithinPartDecisionNumber     0.135      0.036   3.777   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cat("\n--- Pooled EV coefficient without and with the zero control\n")
```

```
##
## --- Pooled EV coefficient without and with the zero control
```

```r
cl_test(NextDelegated ~ EVAlgorithm + WithinPartDecisionNumber, delegation_panel)
```

```
##
## z test of coefficients:
##
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  0.420      0.205   2.053    0.040 *
## EVAlgorithm                 -0.460      0.225  -2.040    0.041 *
## WithinPartDecisionNumber     0.113      0.033   3.386    0.001 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cl_test(NextDelegated ~ EVAlgorithm + Zero + WithinPartDecisionNumber, delegation_panel)
```

```
##
## z test of coefficients:
##
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  0.795      0.210   3.779   <2e-16 ***
## EVAlgorithm                 -0.434      0.233  -1.863    0.062 .
## Zero                        -1.128      0.150  -7.508   <2e-16 ***
## WithinPartDecisionNumber     0.135      0.036   3.776   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cat("\n--- Round 1 only: groups, groups + zero\n")
```

```
##
## --- Round 1 only: groups, groups + zero
```

```r
round1_panel <- filter(delegation_panel, WithinPartDecisionNumber == 1)
cl_test(NextDelegated ~ Group, round1_panel)
```

```
##
## z test of coefficients:
##
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)         0.553      0.263   2.107    0.035 *
## GroupEV_match       0.053      0.394   0.134    0.894
## GroupEV_mismatch   -0.989      0.469  -2.109    0.035 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cl_test(NextDelegated ~ Group + Zero, round1_panel)
```

```
##
## z test of coefficients:
##
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)         0.953      0.313   3.048    0.002 **
## GroupEV_match      -0.109      0.409  -0.266    0.790
```

```
## GroupEV_mismatch   -0.908      0.485  -1.873    0.061 .
## Zero               -1.017      0.381  -2.672    0.008 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat("\n--- Within EV: does a zero weigh more after a mismatched pick?\n")
```

```
##
## --- Within EV: does a zero weigh more after a mismatched pick?
```

```
ev_panel <- delegation_panel %>% filter(EVAlgorithm == 1) %>%
  mutate(Mismatch = 1 * (Group == "EV_mismatch"))
cl_test(NextDelegated ~ Zero * Mismatch + WithinPartDecisionNumber, ev_panel)
```

```
##
## z test of coefficients:
##
##                          Estimate Std. Error z value Pr(>|z|)
## (Intercept)                 0.471      0.233   2.021    0.043 *
## Zero                       -0.845      0.267  -3.164    0.002 **
## Mismatch                    0.262      0.334   0.784    0.433
## WithinPartDecisionNumber    0.083      0.047   1.770    0.077 .
## Zero:Mismatch              -0.729      0.434  -1.680    0.093 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
cat("\n--- Rough mediation arithmetic: zero coefficient x excess zero share\n")
```

```
##
## --- Rough mediation arithmetic: zero coefficient x excess zero share
```

```
b_zero <- coef(glm(NextDelegated ~ Group + Zero + WithinPartDecisionNumber,
                   family = binomial, data = delegation_panel))["Zero"]
zero_share <- tapply(delegation_panel$Zero, delegation_panel$Group, mean)
round(c(EV_match    = unname(b_zero * (zero_share["EV_match"]    - zero_share["Pref"])),
        EV_mismatch = unname(b_zero * (zero_share["EV_mismatch"] - zero_share["Pref"]))),
        ↪  3)
```

```
##    EV_match EV_mismatch
##      -0.003      -0.165
```

```
cat("\n--- With task fixed effects (mismatch is not independent of the task)\n")
```

```
##
## --- With task fixed effects (mismatch is not independent of the task)
```

```
delegation_panel$Task <- factor(delegation_panel$OptionsetID)
cl_test(NextDelegated ~ Group + WithinPartDecisionNumber + Task, delegation_panel)[1:3, ]
```

```
##                  Estimate Std. Error z value Pr(>|z|)
## (Intercept)         0.925      0.279   3.313    0.001
## GroupEV_match      -0.360      0.240  -1.502    0.133
## GroupEV_mismatch   -0.737      0.288  -2.562    0.010
```

```
cl_test(NextDelegated ~ Group + Zero + WithinPartDecisionNumber + Task,
↪  delegation_panel)[2:4, ]
```

```
##                   Estimate Std. Error z value Pr(>|z|)
## GroupEV_match       -0.356      0.249  -1.432    0.152
## GroupEV_mismatch    -0.672      0.295  -2.274    0.023
## Zero                -1.108      0.156  -7.107    0.000
```

```
m_zero_fe <- lm(Zero ~ Group + Task, data = delegation_panel)
cat("Zero share by group, task fixed effects (LPM):\n")
```

```
## Zero share by group, task fixed effects (LPM):
```

```
round(coeftest(m_zero_fe, vcov = vcovCL(m_zero_fe, cluster =
↪  delegation_panel$ProlificID))[2:3, ], 3)
```

```
##                   Estimate Std. Error t value Pr(>|t|)
## GroupEV_match        0.023      0.035   0.643    0.521
## GroupEV_mismatch     0.106      0.051   2.088    0.037
```

```
cat("\n--- Counterfactual for round 1 -> round 2 (EV): what if round-1 EV delegators
↪  whose pick\n",
    "    differed from their preference had continued as often as those whose pick
    ↪  matched?\n")
```

```
##
## --- Counterfactual for round 1 -> round 2 (EV): what if round-1 EV delegators whose pick
##     differed from their preference had continued as often as those whose pick matched?
```

```
r12_cf <- choices %>%
  filter(Part == 2, WithinPartDecisionNumber %in% 1:2) %>%
  mutate(Mismatch = 1 * (Option != PrefAlgoChoice)) %>%
  select(ProlificID, EVAlgorithm, WithinPartDecisionNumber, Delegated, Mismatch) %>%
  pivot_wider(names_from = WithinPartDecisionNumber, values_from = c(Delegated,
  ↪  Mismatch)) %>%
  filter(!is.na(Delegated_1), !is.na(Delegated_2))
cf_stats <- function(d) {
  ev <- d[d$EVAlgorithm == 1, ]; pf <- d[d$EVAlgorithm == 0, ]
  chg <- function(x) mean(x$Delegated_2) - mean(x$Delegated_1)
  del <- ev[ev$Delegated_1 == 1, ]
  cont_match <- mean(del$Delegated_2[del$Mismatch_1 == 0])
  cont_mis   <- mean(del$Delegated_2[del$Mismatch_1 == 1])
  c(EV_change = chg(ev), Pref_change = chg(pf), difference = chg(ev) - chg(pf),
    n_mismatch = sum(del$Mismatch_1 == 1), cont_match = cont_match, cont_mismatch =
    ↪  cont_mis,
    shift_in_EV_round2 = sum(del$Mismatch_1 == 1) * (cont_match - cont_mis) / nrow(ev))
}
round(cf_stats(r12_cf), 3)
```

```
##          EV_change        Pref_change         difference         n_mismatch
##             -0.144             -0.009             -0.135             28.000
##         cont_match      cont_mismatch shift_in_EV_round2
##              0.647              0.393              0.060
```

```
# Participant-level bootstrap interval for the shift (the share of the difference it
# accounts for is too imprecise to report)
set.seed(1)
by_id <- split(r12_cf, r12_cf$ProlificID)
boot_shift <- replicate(2000, cf_stats(do.call(rbind, by_id[sample(names(by_id), replace
↪  = TRUE)]))["shift_in_EV_round2"])
```

```r
round(quantile(boot_shift, c(.025, .5, .975), na.rm = TRUE), 3)
```

```
##  2.5%   50% 97.5%
## 0.004 0.060 0.118
```

4.9.2.1 Forgone payoffs

The feedback screen showed the payoffs of all options in the drawn state, so participants could see what another option would have paid. We compute the gap between the best available option and the realized payoff ("regret", in units of 100 points) and ask whether it predicts next-round delegation beyond a zero payoff. Paper (all rounds 1–7): regret alone is negative (-0.131, $p < 0.001$) but drops to $+0.014$ ($p = 0.74$) once the zero payoff is included. Option 3 of option set 5 never appears in the data, so for that set the best available option may be slightly understated.

```r
payoff_by_set <- split(payoff_matrix, payoff_matrix$OptionsetID)
best_available <- function(set, state) {
  if (is.na(state) || state < 1) return(NA_real_)
  max(payoff_by_set[[as.character(set)]][[state_cols[state]]])
}
regret_panel <- delegation_panel %>%
  mutate(BestAvailable = mapply(best_available, OptionsetID, PayoffRelevantState),
         Regret = (BestAvailable - RelevantPayoff) / 100)

regret_panel %>%
  group_by(Algorithm) %>%
  summarise(n = n(), meanRegret = round(mean(Regret, na.rm = TRUE), 2), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   Algorithm     n meanRegret
##   <chr>     <int>      <dbl>
## 1 EV          413       1.21
## 2 Pref        386       1.65
```

```r
cat("\n--- Regret alone, then regret + zero payoff\n")
```

```
##
## --- Regret alone, then regret + zero payoff
```

```r
cl_test(NextDelegated ~ EVAlgorithm + Regret + WithinPartDecisionNumber, regret_panel)
```

```
##
## z test of coefficients:
##
##                           Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  0.632      0.214   2.947    0.003 **
## EVAlgorithm                 -0.529      0.228  -2.322    0.020 *
## Regret                      -0.131      0.035  -3.806   <2e-16 ***
## WithinPartDecisionNumber     0.118      0.034   3.490   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
cl_test(NextDelegated ~ EVAlgorithm + Zero + Regret + WithinPartDecisionNumber,
↪  regret_panel)
```

```
##
## z test of coefficients:
##
```

```
##                            Estimate Std. Error z value Pr(>|z|)
## (Intercept)                   0.784      0.215   3.642   <2e-16 ***
## EVAlgorithm                  -0.427      0.235  -1.814    0.070 .
## Zero                         -1.159      0.178  -6.517   <2e-16 ***
## Regret                        0.014      0.042   0.329    0.742
## WithinPartDecisionNumber      0.135      0.036   3.777   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

4.9.2.2 Percentage-point version of the match/mismatch comparison The paper reports these comparisons as percentage-point differences from linear probability models with participant-clustered standard errors. Paper: matching EV picks −8.4 points (95% CI [−18.7, 2.0], $p = 0.112$) and mismatching EV picks −14.5 points (95% CI [−26.9, −2.0], $p = 0.023$) relative to preference-based delegation; mismatching vs. matching EV picks −6.1 points ($p = 0.266$); with task fixed effects −16.4 points ($p = 0.012$) for mismatching picks; controlling for a zero payoff −10.8 points ($p = 0.078$); a zero payoff within EV lowers continued delegation by 20.0 points after a matching and by 36.7 points after a mismatching pick (difference 16.7 points, $p = 0.083$).

```
lpm_pp <- function(formula, data, terms) {
  m  <- lm(formula, data = data)
  V  <- vcovCL(m, cluster = data$ProlificID)
  ct <- coeftest(m, vcov = V)
  ci <- coefci(m, vcov. = V)
  round(cbind(pp = 100 * ct[terms, 1], lo = 100 * ci[terms, 1], hi = 100 * ci[terms, 2],
              p = ct[terms, 4]), 3)
}
delegation_panel$Task <- factor(delegation_panel$OptionsetID)
lpm_pp(NextDelegated ~ Group + WithinPartDecisionNumber, delegation_panel,
       c("GroupEV_match", "GroupEV_mismatch"))
```

```
##                         pp      lo     hi     p
## GroupEV_match     -8.388 -18.740  1.965 0.112
## GroupEV_mismatch -14.481 -26.947 -2.015 0.023
```

```
lpm_pp(NextDelegated ~ Group + WithinPartDecisionNumber + Task, delegation_panel,
       c("GroupEV_match", "GroupEV_mismatch"))
```

```
##                         pp      lo     hi     p
## GroupEV_match     -7.806 -18.231  2.620 0.142
## GroupEV_mismatch -16.412 -29.157 -3.667 0.012
```

```
lpm_pp(NextDelegated ~ Group + Zero + WithinPartDecisionNumber, delegation_panel,
       c("GroupEV_match", "GroupEV_mismatch", "Zero"))
```

```
##                         pp      lo      hi     p
## GroupEV_match     -8.277 -18.451   1.898 0.111
## GroupEV_mismatch -10.773 -22.766   1.220 0.078
## Zero             -25.018 -31.704 -18.331 0.000
```

```
cat("\nMismatching vs. matching EV picks\n")
```

```
##
## Mismatching vs. matching EV picks
```

```
delegation_panel$GroupRef <- relevel(delegation_panel$Group, ref = "EV_match")
lpm_pp(NextDelegated ~ GroupRef + WithinPartDecisionNumber, delegation_panel,
↪  "GroupRefEV_mismatch")
```

```
##          pp      lo    hi     p
## [1,] -6.093 -16.845 4.659 0.266
```

```
cat("\nWithin EV: zero payoff x mismatching pick\n")
```

```
##
## Within EV: zero payoff x mismatching pick
```

```
ev_lpm <- delegation_panel %>% filter(EVAlgorithm == 1) %>%
  mutate(Mismatch = 1 * (Group == "EV_mismatch"))
lpm_pp(NextDelegated ~ Zero * Mismatch + WithinPartDecisionNumber, ev_lpm,
       c("Zero", "Mismatch", "Zero:Mismatch"))
```

```
##                    pp      lo     hi     p
## Zero          -20.003 -32.476 -7.529 0.002
## Mismatch        5.302  -7.703 18.308 0.423
## Zero:Mismatch -16.653 -35.513  2.208 0.083
```

```
cat("\nPooled EV gap without and with the zero control\n")
```

```
##
## Pooled EV gap without and with the zero control
```

```
lpm_pp(NextDelegated ~ EVAlgorithm + WithinPartDecisionNumber, delegation_panel,
↪  "EVAlgorithm")
```

```
##           pp      lo    hi    p
## [1,] -10.337 -20.213 -0.46 0.04
```

```
lpm_pp(NextDelegated ~ EVAlgorithm + Zero + WithinPartDecisionNumber, delegation_panel,
       c("EVAlgorithm", "Zero"))
```

```
##                   pp      lo      hi     p
## EVAlgorithm   -9.066 -18.667   0.534 0.064
## Zero         -25.189 -31.775 -18.602 0.000
```

4.9.3 (iii) The first-contact gap sits at the opt-out margin

The framework predicts that resistance is concentrated in participants who value choice authority and opt out categorically. We first compare round-1 delegation among participants who delegate at least once. Paper: 65.6% (Pref) vs. 69.3% (EV), one-sided Boschloo $p = 0.289$. Because never-delegation is defined over all eight rounds and thus includes round 1, we then repeat the comparison with "otherwise-never-delegators", defined without the round(s) under analysis: for round 1, participants who never delegate in rounds 2–8; for the rounds 1–2 analysis, participants who never delegate in rounds 3–8. Paper (round 1): 16.8% (Pref) vs. 7.6% (EV), one-sided Boschloo $p = 0.019$; round-1 delegation among all other participants 64.9% vs. 67.9% (one-sided $p = 0.330$). The comparisons are descriptive, since they condition on later behaviour. We do not report a numerical split of the 11.2-point gap, because the within-group part rests on only 28 otherwise-never-delegators.

```
wide <- choices %>%
  filter(Part == 2) %>%
  select(ProlificID, EVAlgorithm, WithinPartDecisionNumber, Delegated) %>%
  pivot_wider(names_from = WithinPartDecisionNumber, values_from = Delegated,
              names_prefix = "d")
dmat <- as.matrix(wide[, paste0("d", 1:8)])
```

```r
# 1 if the participant never delegates in the given rounds (NA if no valid decision
↪  there)
never_in <- function(cols) {
  x <- dmat[, cols, drop = FALSE]
  ifelse(rowSums(!is.na(x)) == 0, NA, 1 * (rowSums(x, na.rm = TRUE) == 0))
}
wide$Never              <- never_in(1:8)   # standard definition (all rounds)
wide$OtherwiseNever_r1  <- never_in(2:8)   # for analyses of round 1
wide$OtherwiseNever_r12 <- never_in(3:8)   # for analyses of rounds 1 and 2
wide$lateRate <- rowSums(dmat[, 2:8, drop = FALSE], na.rm = TRUE) /
                 rowSums(!is.na(dmat[, 2:8, drop = FALSE]))

# Boschloo exact test on a 2 x 2 table built from two groups (Pref = 0, EV = 1)
boschloo_p <- function(outcome, group, alternative = "two.sided") {
  tab <- table(factor(group, levels = c(0, 1)), factor(outcome, levels = c(1, 0)))
  exact.test(tab, method = "Boschloo", alternative = alternative, to.plot =
  ↪  FALSE)$p.value
}

cat("--- Plain definition: participants who delegate at least once\n")
```

```
## --- Plain definition: participants who delegate at least once
```

```r
not_never <- filter(wide, Never == 0, !is.na(d1))
not_never %>%
  group_by(EVAlgorithm) %>%
  summarise(n = n(), round1 = round(100 * mean(d1), 1),
            rounds2to8 = round(100 * mean(lateRate, na.rm = TRUE), 1), .groups = "drop")
```

```
## # A tibble: 2 x 4
##   EVAlgorithm     n round1 rounds2to8
##         <int> <int>  <dbl>      <dbl>
## 1           0    96   65.6       56.4
## 2           1   114   69.3       49.8
```

```r
# one-sided: EV delegates more than Pref ("less" = first row, Pref, has the lower share)
cat("Round-1 delegation, EV > Pref (one-sided Boschloo p):",
    round(boschloo_p(not_never$d1, not_never$EVAlgorithm, "less"), 3), "\n")
```

```
## Round-1 delegation, EV > Pref (one-sided Boschloo p): 0.289
```

```r
cat("\n--- Robustness: otherwise-never-delegators (never in rounds 2-8)\n")
```

```
##
## --- Robustness: otherwise-never-delegators (never in rounds 2-8)
```

```r
A <- filter(wide, !is.na(d1), !is.na(OtherwiseNever_r1))
A %>%
  group_by(EVAlgorithm) %>%
  summarise(N = n(),
            nOtherwiseNever       = sum(OtherwiseNever_r1),
            pctOtherwiseNever     = round(100 * mean(OtherwiseNever_r1), 1),
            round1_others         = round(100 * mean(d1[OtherwiseNever_r1 == 0]), 1),
            round1_otherwiseNever = round(100 * mean(d1[OtherwiseNever_r1 == 1]), 1),
            .groups = "drop")
```

```
## # A tibble: 2 x 6
##   EVAlgorithm     N nOtherwiseNever pctOtherwiseNever round1_others
##         <int> <int>           <dbl>             <dbl>         <dbl>
## 1           0   113              19              16.8          64.9
## 2           1   118               9               7.6          67.9
## # i 1 more variable: round1_otherwiseNever <dbl>
```

```
cat("Share of otherwise-never-delegators, Pref > EV (one-sided Boschloo p):",
    round(boschloo_p(A$OtherwiseNever_r1, A$EVAlgorithm, "greater"), 3), "\n")
```

```
## Share of otherwise-never-delegators, Pref > EV (one-sided Boschloo p): 0.019
```

```
cat("  two-sided:", round(boschloo_p(A$OtherwiseNever_r1, A$EVAlgorithm), 3), "\n")
```

```
##   two-sided: 0.037
```

```
others1 <- filter(A, OtherwiseNever_r1 == 0)
cat("Round-1 delegation among all others, EV > Pref (one-sided Boschloo p):",
    round(boschloo_p(others1$d1, others1$EVAlgorithm, "less"), 3), "\n")
```

```
## Round-1 delegation among all others, EV > Pref (one-sided Boschloo p): 0.33
```

4.9.4 Does the round-1 to round-2 decline differ between conditions?

Descriptively the EV decline is larger, but the difference-in-differences is imprecise. Paper: round 1 vs. round 2 interaction clustered $p = 0.14$; round 1 vs. rounds 2–8 interaction $p = 0.050$. We report both as exploratory and do not read the first as evidence that the two declines are equal. Among participants who delegate at least once in rounds 3–8 (the analysed rounds do not enter this definition), the round 1 to round 2 change is -5.8 points (Pref) vs. -14.5 points (EV); the interaction is -0.36 ($p = 0.38$).

```
round12_all <- choices %>%
  filter(Part == 2, WithinPartDecisionNumber %in% 1:2) %>%
  mutate(Round2 = 1 * (WithinPartDecisionNumber == 2))
cl_test(Delegated ~ EVAlgorithm * Round2, round12_all)
```

```
##
## z test of coefficients:
##
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)            0.231      0.190   1.218    0.223
## EVAlgorithm            0.475      0.273   1.740    0.082 .
## Round2                -0.091      0.236  -0.384    0.701
## EVAlgorithm:Round2    -0.514      0.349  -1.472    0.141
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
round1_vs_rest <- choices %>%
  filter(Part == 2) %>%
  mutate(Round1 = 1 * (WithinPartDecisionNumber == 1))
cl_test(Delegated ~ EVAlgorithm * Round1, round1_vs_rest)
```

```
##
## z test of coefficients:
##
##                     Estimate Std. Error z value Pr(>|z|)
## (Intercept)           -0.100      0.137  -0.733    0.463
## EVAlgorithm            0.015      0.179   0.083    0.934
```

```
## Round1                0.331      0.153   2.173    0.030 *
## EVAlgorithm:Round1    0.460      0.235   1.960    0.050 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Share of otherwise-never-delegators (rounds 3-8) and the decline among all others
E <- filter(wide, !is.na(d1), !is.na(d2), !is.na(OtherwiseNever_r12))
E %>%
  group_by(EVAlgorithm) %>%
  summarise(N = n(), pctOtherwiseNever = round(100 * mean(OtherwiseNever_r12), 1),
            .groups = "drop")
```

```
## # A tibble: 2 x 3
##   EVAlgorithm     N pctOtherwiseNever
##         <int> <int>             <dbl>
## 1           0   112              23.2
## 2           1   118              12.7
```

```
cat("Share of otherwise-never-delegators (rounds 3-8), Pref > EV (one-sided Boschloo
↪  p):",
    round(boschloo_p(E$OtherwiseNever_r12, E$EVAlgorithm, "greater"), 3), "\n")
```

```
## Share of otherwise-never-delegators (rounds 3-8), Pref > EV (one-sided Boschloo p): 0.022
```

```
others12 <- filter(E, OtherwiseNever_r12 == 0)
others12 %>%
  group_by(EVAlgorithm) %>%
  summarise(n = n(), round1 = round(100 * mean(d1), 1), round2 = round(100 * mean(d2),
  ↪  1),
            change = round(round2 - round1, 1), .groups = "drop")
```

```
## # A tibble: 2 x 5
##   EVAlgorithm     n round1 round2 change
##         <int> <int>  <dbl>  <dbl>  <dbl>
## 1           0    86   68.6   62.8   -5.8
## 2           1   103   68.9   54.4  -14.5
```

```
others12_long <- others12 %>%
  select(ProlificID, EVAlgorithm, d1, d2) %>%
  pivot_longer(c(d1, d2), names_to = "round", values_to = "Delegated") %>%
  mutate(Round2 = 1 * (round == "d2"))
cl_test(Delegated ~ EVAlgorithm * Round2, others12_long)
```

```
##
## z test of coefficients:
##
##                    Estimate Std. Error z value Pr(>|z|)
## (Intercept)           0.782      0.233   3.355    0.001 ***
## EVAlgorithm           0.015      0.316   0.048    0.962
## Round2               -0.258      0.306  -0.844    0.399
## EVAlgorithm:Round2   -0.363      0.412  -0.881    0.378
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Does the round-1 choice predict later delegation differently by condition?
later <- choices %>%
```

```
  filter(Part == 2, WithinPartDecisionNumber >= 2) %>%
  left_join(choices %>% filter(Part == 2, WithinPartDecisionNumber == 1) %>%
              select(ProlificID, D1 = Delegated), by = "ProlificID") %>%
  filter(!is.na(D1))
later %>% group_by(Algorithm, D1) %>%
  summarise(n = n(), delegationRate = round(mean(Delegated), 3), .groups = "drop")
```

```
## # A tibble: 4 x 4
##   Algorithm    D1     n delegationRate
##   <chr>     <int> <int>          <dbl>
## 1 EV            0   271          0.284
## 2 EV            1   548          0.575
## 3 Pref          0   349          0.229
## 4 Pref          1   435          0.678
```

```
cl_test(Delegated ~ D1 * EVAlgorithm, later)
```

```
##
## z test of coefficients:
##
##                Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -1.213      0.191  -6.336   <2e-16 ***
## D1                1.958      0.268   7.317   <2e-16 ***
## EVAlgorithm       0.289      0.243   1.188    0.235
## D1:EVAlgorithm   -0.732      0.339  -2.159    0.031 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

5 Categorical non-delegation

We classify participants as never-, sometimes-, or always-delegators and compare never-delegation across conditions (paper: 15.7% Pref vs. 3.4% EV).

```
choices <- choices %>%
  group_by(ProlificID) %>%
  mutate(sumDelegations = sum(Delegated)) %>%
  ungroup() %>%
  left_join(
    prolificData %>% select(Participant.id, TimeInSeconds = Time.taken),
    by = c("ProlificID" = "Participant.id")
  )

analysis_data <- choices %>%
  group_by(ProlificID) %>%
  summarise(
    EVAlgorithm                = first(EVAlgorithm),
    sumDelegations             = first(sumDelegations),
    Female                     = first(Female),
    Age                        = first(Age),
    HHIncome                   = first(HHIncome),
    Education                  = first(Education),
    ProbOfBadComputerChoices   = first(ProbOfBadComputerChoices),
    ProbOfAbsurdComputerChoices = first(ProbOfAbsurdComputerChoices),
    ComputerRiskTaking         = first(ComputerRiskTaking),
    PBetChoices                = first(PBet_count),
    TimeInSeconds              = first(TimeInSeconds),
    .groups = "drop"
  ) %>%
  mutate(
    DelegationGroup = ifelse(sumDelegations == 0, "Never", "AtLeastOnce"),
    NonDelegator    = 1 * (sumDelegations == 0),
    DelegatorType   = factor(
      if_else(sumDelegations == 0, "NeverDelegator",
              if_else(sumDelegations == 8, "AlwaysDelegator", "SometimesDelegator")),
      levels = c("NeverDelegator", "SometimesDelegator", "AlwaysDelegator")
    )
  ) %>%
  left_join(
    subjects %>% select(ProlificID, RiskTaking, FinMarketExperience,
                        FinanciallyResponsible, AIXperience, Reliable),
    by = "ProlificID"
  )

# Distribution of delegator types by condition (column 2 = EV)
table(analysis_data$DelegatorType, analysis_data$EVAlgorithm)
```

```
##
##                       0  1
##   NeverDelegator     18  4
##   SometimesDelegator 75 98
##   AlwaysDelegator    22 16
```

```r
prop.table(table(analysis_data$DelegatorType, analysis_data$EVAlgorithm), margin = 2)
```

```
##
##                              0          1
##   NeverDelegator     0.15652174 0.03389831
##   SometimesDelegator 0.65217391 0.83050847
##   AlwaysDelegator    0.19130435 0.13559322
```

5.1 Significance test and effect size for categorical non-delegation

Main test: one-sided Boschloo exact test (Pref > EV) with the two treatment arms as the binomial groups. The outcome was not preregistered; a one-sided test is in line with the general prediction that personalization reduces acceptance, and it is the same test as for the earlier study in the footnote. The two-sided Boschloo test, Fisher's exact test (used for the odds ratio and its CI), and the risk difference with Wald 95% CI are reported alongside (paper: one-sided $p < 0.001$, two-sided $p = 0.001$; RD = 12.3pp [4.9, 19.7]; OR = 5.25 [1.65, 22.07]).

```r
# Rows: treatment arms (the binomial groups); columns: never-delegator yes / no
nondeleg_tab <- table(
  Treatment    = factor(analysis_data$EVAlgorithm, levels = c(0, 1),
                        labels = c("Pref", "EV")),
  NonDelegator = factor(analysis_data$NonDelegator, levels = c(1, 0),
                        labels = c("Never", "NotNever"))
)
nondeleg_tab
```

```
##           NonDelegator
## Treatment Never NotNever
##      Pref    18       97
##      EV       4      114
```

```r
# Main test: one-sided Boschloo (Pref > EV)
exact.test(nondeleg_tab, method = "Boschloo", alternative = "greater",
           to.plot = FALSE)$p.value
```

```
## [1] 0.0008202151
```

```r
# Two-sided, for reference
exact.test(nondeleg_tab, method = "Boschloo", to.plot = FALSE)$p.value
```

```
## [1] 0.001149108
```

```r
# Odds ratio and its CI (Fisher's exact; expect OR ~ 5.25)
fisher.test(nondeleg_tab)
```

```
##
##  Fisher's Exact Test for Count Data
##
## data:  nondeleg_tab
## p-value = 0.001436
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
##   1.654958 22.069431
## sample estimates:
## odds ratio
##   5.254962
```

```r
# Risk difference + Wald 95% CI
p_pref_nd <- nondeleg_tab["Pref", "Never"] / sum(nondeleg_tab["Pref", ])
p_ev_nd   <- nondeleg_tab["EV",   "Never"] / sum(nondeleg_tab["EV", ])
n_pref_nd <- sum(nondeleg_tab["Pref", ])
n_ev_nd   <- sum(nondeleg_tab["EV", ])
rd_nd     <- p_pref_nd - p_ev_nd
se_rd_nd  <- sqrt(p_pref_nd * (1 - p_pref_nd) / n_pref_nd +
                  p_ev_nd * (1 - p_ev_nd) / n_ev_nd)
cat("Risk difference:", round(rd_nd, 3), "\n")
```

```
## Risk difference: 0.123
```

```r
cat("Wald 95% CI:", round(rd_nd - 1.96 * se_rd_nd, 3), "to",
    round(rd_nd + 1.96 * se_rd_nd, 3), "\n")
```

```
## Wald 95% CI: 0.049 to 0.197
```

5.2 Comparisons of never-delegators vs. delegators

The tests below are in the order in which they appear in the text (Results, "Categorical non-delegation"); never-delegators are compared with participants who delegated at least once. Tests that are *not* reported in the text are collected in a separate subsection further below.

```r
# Age (the text lists never-delegators first: 41.8 vs. 42.9, t = -0.329)
t.test(Age ~ factor(DelegationGroup, levels = c("Never", "AtLeastOnce")),
       data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  Age by factor(DelegationGroup, levels = c("Never", "AtLeastOnce"))
## t = -0.32876, df = 24.878, p-value = 0.7451
## alternative hypothesis: true difference in means between group Never and group AtLeastOnce is not eq
## 95 percent confidence interval:
##  -7.849952  5.689245
## sample estimates:
##       mean in group Never mean in group AtLeastOnce
##                  41.77273                  42.85308
```

```r
# Household income (categorical)
chisq.test(table(analysis_data$DelegationGroup, analysis_data$HHIncome))
```

```
##
##  Pearson's Chi-squared test
##
## data:  table(analysis_data$DelegationGroup, analysis_data$HHIncome)
## X-squared = 13.171, df = 11, p-value = 0.2823
```

```r
# Education (categorical)
chisq.test(table(analysis_data$DelegationGroup, analysis_data$Education))
```

```
##
##  Pearson's Chi-squared test
##
## data:  table(analysis_data$DelegationGroup, analysis_data$Education)
## X-squared = 1.859, df = 5, p-value = 0.8683
```

```r
# Time in experiment (delegators vs. never-delegators; t = -0.971)
t.test(TimeInSeconds ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  TimeInSeconds by NonDelegator
## t = -0.9713, df = 25.374, p-value = 0.3406
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -385.8898  138.4297
## sample estimates:
## mean in group 0 mean in group 1
##        1113.043        1236.773
```

```r
# Gender: never-delegation rate among males vs. females (Boschloo; text: p = 0.156)
female_table <- table(
  DelegationGroup = analysis_data$DelegationGroup,
  Female          = analysis_data$Female
)
female_table
```

```
##                Female
## DelegationGroup  -1   0 0.5   1
##     AtLeastOnce   1 103   2 105
##     Never         0  14   1   7
```

```r
exact.test(female_table[, c("0", "1")], method = "Boschloo", to.plot = FALSE)$p.value
```

```
## [1] 0.1561955
```

```r
# Self-reported willingness to take financial risks
t.test(RiskTaking ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  RiskTaking by NonDelegator
## t = -0.22379, df = 24.99, p-value = 0.8247
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -1.540832  1.238807
## sample estimates:
## mean in group 0 mean in group 1
##        4.530806        4.681818
```

```r
# Self-reported response reliability / care
t.test(Reliable ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  Reliable by NonDelegator
## t = -0.058833, df = 25.559, p-value = 0.9535
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
```

```
## 95 percent confidence interval:
##  -0.1394700  0.1317147
## sample estimates:
## mean in group 0 mean in group 1
##        6.905213        6.909091
```

```
# Frequency of AI use / interactions
t.test(AIXperience ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  AIXperience by NonDelegator
## t = 0.86582, df = 24.274, p-value = 0.3951
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -0.9475606  2.3185214
## sample estimates:
## mean in group 0 mean in group 1
##        5.867299        5.181818
```

Within the preference-based condition: Part-1 satisfaction by delegation type (this chunk also creates `subjects2`, which is used in the section on reasons below).

```
NeverDelegate <- choices %>%
  select(ProlificID, sumDelegations) %>%
  distinct() %>%
  mutate(sumDeleg0 = (sumDelegations == 0))

subjects2 <- subjects %>% left_join(NeverDelegate, by = "ProlificID")

subjects2 %>%
  filter(EVAlgorithm == 0) %>%
  group_by(sumDeleg0) %>%
  summarise(SatisfiedPart1 = mean(SatisfiedPart1, na.rm = TRUE),
            n = n(), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   sumDeleg0 SatisfiedPart1     n
##   <lgl>              <dbl> <int>
## 1 FALSE               7.56    97
## 2 TRUE                7.94    18
```

```
t.test(SatisfiedPart1 ~ sumDeleg0, data = filter(subjects2, EVAlgorithm == 0))
```

```
##
##  Welch Two Sample t-test
##
## data:  SatisfiedPart1 by sumDeleg0
## t = -1.3996, df = 39.691, p-value = 0.1694
## alternative hypothesis: true difference in means between group FALSE and group TRUE is not equal to 0
## 95 percent confidence interval:
##  -0.9478024  0.1723156
## sample estimates:
## mean in group FALSE  mean in group TRUE
##            7.556701            7.944444
```

5.2.1 Non-parametric robustness check (Wilcoxon), as in the footnote

Same comparisons, same order (paper: age $p = 0.649$; time $p = 0.105$; risk-taking $p = 0.854$; reliability $p = 0.956$; AI experience $p = 0.445$).

```
wilcox.test(Age ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  Age by NonDelegator
## W = 2458.5, p-value = 0.6487
## alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(TimeInSeconds ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  TimeInSeconds by NonDelegator
## W = 1833, p-value = 0.1052
## alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(RiskTaking ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  RiskTaking by NonDelegator
## W = 2265.5, p-value = 0.854
## alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(Reliable ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  Reliable by NonDelegator
## W = 2312, p-value = 0.9555
## alternative hypothesis: true location shift is not equal to 0
```

```
wilcox.test(AIXperience ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  AIXperience by NonDelegator
## W = 2550, p-value = 0.445
## alternative hypothesis: true location shift is not equal to 0
```

5.2.2 Tests that are not reported in the text

Kept for completeness; none of these appears in the manuscript.

```
# Distributional test on time in experiment
ks.test(TimeInSeconds ~ NonDelegator, data = analysis_data)
```

```
##
```

```
##  Exact two-sample Kolmogorov-Smirnov test
##
## data:  TimeInSeconds by NonDelegator
## D = 0.28264, p-value = 0.06603
## alternative hypothesis: two-sided
```

```
# Share of household financial decisions the participant is responsible for
t.test(FinanciallyResponsible ~ NonDelegator, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  FinanciallyResponsible by NonDelegator
## t = 1.229, df = 24.512, p-value = 0.2307
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -0.6803629  2.6889799
## sample estimates:
## mean in group 0 mean in group 1
##        6.549763        5.545455
```

```
wilcox.test(FinanciallyResponsible ~ NonDelegator, data = analysis_data)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  FinanciallyResponsible by NonDelegator
## W = 2677, p-value = 0.2302
## alternative hypothesis: true location shift is not equal to 0
```

```
# Part-1 satisfaction (Pref condition only), non-parametric
wilcox.test(SatisfiedPart1 ~ sumDeleg0, data = filter(subjects2, EVAlgorithm == 0))
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  SatisfiedPart1 by sumDeleg0
## W = 795, p-value = 0.5365
## alternative hypothesis: true location shift is not equal to 0
```

5.3 Choice quality among never-delegators (Limitations)

Tests whether never-delegators' non-dominated choice rate among their self-made Part-2 decisions differs from that of participants who delegated at least once, used in the manuscript to argue against a task-difficulty confound for categorical non-delegation (paper: 34.7% vs. 33.6%; Welch $t = -0.25$, $p = 0.806$; Wilcoxon $p = 0.669$).

Note on the sample sizes. The first table counts *decisions* and the second *participants*, so the numbers differ by construction. The 22 never-delegators made 175 self-chosen decisions in total (22 x 8 = 176 minus one round without a valid decision). The 170 in the participant-level comparison is the number of participants who delegated at least once *and* made at least one valid decision themselves: of the 211 participants who delegated at least once, 40 delegated in every valid round (38 in all 8 rounds, 2 with fewer valid rounds) and 1 has only a timed-out own decision, so they contribute no self-chosen decision (211 - 41 = 170).

```
selfchoice <- choices %>%
  filter(Part == 2, Delegated == 0, Option > 0) %>%
```

```r
  left_join(
    analysis_data %>% select(ProlificID, DelegatorType, DelegationGroup),
    by = "ProlificID"
  )

# Decision level, by delegator type (n_decisions = self-chosen decisions,
# n_participants = participants contributing to them)
selfchoice %>%
  group_by(DelegatorType) %>%
  summarise(avNonDom = mean(Nondominated, na.rm = TRUE),
            n_decisions = n(),
            n_participants = n_distinct(ProlificID), .groups = "drop")
```

```
## # A tibble: 2 x 4
##   DelegatorType      avNonDom n_decisions n_participants
##   <fct>                 <dbl>       <int>          <int>
## 1 NeverDelegator        0.349         175             22
## 2 SometimesDelegator    0.345         753            170
```

```r
# Participant level: average non-dominated rate, Never vs. AtLeastOnce
partavg <- selfchoice %>%
  group_by(ProlificID, DelegationGroup) %>%
  summarise(Nondominated = mean(Nondominated, na.rm = TRUE), .groups = "drop")

partavg %>%
  group_by(DelegationGroup) %>%
  summarise(Nondominated = mean(Nondominated), n_participants = n(), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   DelegationGroup Nondominated n_participants
##   <chr>                  <dbl>          <int>
## 1 AtLeastOnce            0.336            170
## 2 Never                  0.347             22
```

```r
t.test(Nondominated ~ DelegationGroup, data = partavg)
```

```
##
##  Welch Two Sample t-test
##
## data:  Nondominated by DelegationGroup
## t = -0.248, df = 34.608, p-value = 0.8056
## alternative hypothesis: true difference in means between group AtLeastOnce and group Never is not eq
## 95 percent confidence interval:
##  -0.09505609  0.07436727
## sample estimates:
## mean in group AtLeastOnce       mean in group Never
##                 0.3362465                 0.3465909
```

```r
wilcox.test(Nondominated ~ DelegationGroup, data = partavg)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  Nondominated by DelegationGroup
## W = 1765.5, p-value = 0.6689
```

```
## alternative hypothesis: true location shift is not equal to 0
```

6 Responses to feedback

6.1 Preregistered secondary hypothesis: switching to non-delegation after a zero payoff

The preregistration (question 8) contains one secondary hypothesis: after delegating to the algorithm with a 0-payoff, participants in PREF are more likely to switch to non-delegation in the following round than under any other circumstances, in particular than under the same circumstances in the EV condition. The preregistration does not specify a test. We estimate an LPM of switching to non-delegation in rounds 2–8 that follow a delegated round, with task and decision-number fixed effects and participant-clustered SEs; the preregistered prediction corresponds to a *positive* `PREF x ZeroPayoff` interaction (tested one-sided). Note that lags are taken over the available rows of a participant (16 of the 1,864 Part-2 rounds have no valid decision and are dropped, which affects 13 participants).

```
switchdat <- choices %>%
  filter(Part == 2) %>%
  group_by(ProlificID) %>%
  arrange(WithinPartDecisionNumber, .by_group = TRUE) %>%
  mutate(lagDelegated = lag(Delegated, 1),
         lagPayoff    = lag(RelevantPayoff, 1)) %>%
  ungroup() %>%
  filter(WithinPartDecisionNumber > 1, !is.na(lagDelegated)) %>%
  mutate(PREF         = 1 - EVAlgorithm,
         ZeroPayoff   = 1 * (lagPayoff == 0),
         SwitchToNon  = 1 * (lagDelegated == 1 & Delegated == 0))

# Rounds that follow a delegated round (paper: 799 observations, 209 participants)
afterDeleg <- filter(switchdat, lagDelegated == 1)
c(N = nrow(afterDeleg), participants = n_distinct(afterDeleg$ProlificID))
```

```
##            N participants
##          799          209
```

```
# Raw switching rates (paper: EV 30.3% / 55.6%; PREF 21.5% / 45.3%)
afterDeleg %>%
  group_by(PREF, ZeroPayoff) %>%
  summarise(rate = mean(SwitchToNon), n = n(), .groups = "drop")
```

```
## # A tibble: 4 x 4
##    PREF ZeroPayoff  rate     n
##   <dbl>      <dbl> <dbl> <int>
## 1     0          0 0.303   244
## 2     0          1 0.556   169
## 3     1          0 0.215   247
## 4     1          1 0.453   139
```

```
switchReg <- feols(
  SwitchToNon ~ PREF * ZeroPayoff | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID, data = afterDeleg
)
summary(switchReg)
```

```
## OLS estimation, Dep. Var.: SwitchToNon
## Observations: 799
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
```

```
##                  Estimate Std. Error   t value   Pr(>|t|)
## PREF            -0.081633   0.050307 -1.622718 1.0616e-01
## ZeroPayoff       0.263179   0.047966  5.486745 1.1829e-07 ***
## PREF:ZeroPayoff -0.022773   0.066894 -0.340427 7.3388e-01
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.45389     Adj. R2: 0.082376
##                 Within R2: 0.078113
```

```r
# One-sided p-value for the preregistered direction (interaction > 0);
# paper: beta = -0.023, SE = 0.067, one-sided p = 0.633 (two-sided 0.734)
ct <- coeftable(switchReg)
pt(ct["PREF:ZeroPayoff", "t value"],
   df = n_distinct(afterDeleg$ProlificID) - 1, lower.tail = FALSE)
```

```
## [1] 0.6330606
```

Supplementary contrasts reported in the text: (a) only rounds following a zero-payoff delegation, PREF vs. EV (paper: beta = -0.105, SE = 0.070, two-sided p = 0.140); (b) the zero-payoff effect within each condition (paper: 0.254 in EV, 0.250 in PREF, both $p < 0.001$); (c) comparison with *all* other rounds 2–8 as the reference (paper: interaction beta = -0.071, SE = 0.071, one-sided p = 0.840; 1,615 observations).

```r
# (a) after zero-payoff delegation only
feols(SwitchToNon ~ PREF | OptionsetID + WithinPartDecisionNumber,
      cluster = ~ProlificID, data = filter(afterDeleg, ZeroPayoff == 1)) %>% summary()
```

```
## OLS estimation, Dep. Var.: SwitchToNon
## Observations: 308
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##       Estimate Std. Error  t value Pr(>|t|)
## PREF -0.104583   0.070485 -1.48377  0.13966
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.48052     Adj. R2: 0.031905
##                 Within R2: 0.011297
```

```r
# (b) zero vs. positive payoff after delegation, by condition
feols(SwitchToNon ~ ZeroPayoff | OptionsetID + WithinPartDecisionNumber,
      cluster = ~ProlificID, data = filter(afterDeleg, PREF == 0)) %>% summary()  # EV
```

```
## OLS estimation, Dep. Var.: SwitchToNon
## Observations: 413
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##            Estimate Std. Error t value   Pr(>|t|)
## ZeroPayoff 0.254466   0.047696 5.33518 4.9674e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.469043     Adj. R2: 0.056232
##                  Within R2: 0.064522
```

```r
feols(SwitchToNon ~ ZeroPayoff | OptionsetID + WithinPartDecisionNumber,
      cluster = ~ProlificID, data = filter(afterDeleg, PREF == 1)) %>% summary()  # PREF
```

```
## OLS estimation, Dep. Var.: SwitchToNon
```

```
## Observations: 386
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##             Estimate Std. Error t value   Pr(>|t|)
## ZeroPayoff 0.250368   0.047963 5.22005 1.0669e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.43194     Adj. R2: 0.078944
##                 Within R2: 0.069548
```

```
# (c) "any other circumstances" = all rounds 2--8
switchAll <- switchdat %>% mutate(ZeroAfterDeleg = 1 * (lagDelegated == 1 & lagPayoff ==
↪  0))
switchRegAll <- feols(
  SwitchToNon ~ PREF * ZeroAfterDeleg | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID, data = switchAll
)
summary(switchRegAll)
```

```
## OLS estimation, Dep. Var.: SwitchToNon
## Observations: 1,615
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                      Estimate Std. Error   t value  Pr(>|t|)
## PREF                -0.032976   0.016620 -1.984080  0.048426 *
## ZeroAfterDeleg       0.442525   0.048412  9.140855 < 2.2e-16 ***
## PREF:ZeroAfterDeleg -0.070832   0.071050 -0.996930  0.319837
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.340042     Adj. R2: 0.194173
##                  Within R2: 0.187901
```

```
ctA <- coeftable(switchRegAll)
pt(ctA["PREF:ZeroAfterDeleg", "t value"],
   df = n_distinct(switchAll$ProlificID) - 1, lower.tail = FALSE)
```

```
## [1] 0.8400814
```

6.2 Exploratory: win-stay / lose-shift

Not preregistered. Win-stay/lose-shift model on rounds 2–8 (paper, Section "Responses to feedback"), and its decomposition by who produced the previous outcome.

```
choices <- choices %>%
  group_by(ProlificID) %>%
  arrange(WithinPartDecisionNumber, .by_group = TRUE) %>%
  mutate(
    lagDelegated          = lag(Delegated, 1),
    lagRelevantPayoff     = lag(RelevantPayoff, 1),
    lagSatisfactionRating = lag(SatisfactionRating, 1)
  ) %>%
  ungroup()

choices$stay <- 1 * (choices$Delegated == choices$lagDelegated)
# The paper models SWITCHING (= 1 - stay), so that a positive coefficient always means
↪  more switching
```

```r
choices$switch <- 1 - choices$stay

# "Prior win": the outcome of the PREVIOUS round was strictly positive.
# (lagWin is defined from the lagged payoff, i.e. the genuine win-stay/lose-shift
# notion of reacting to the previously experienced outcome.)
choices$lagWin <- 1 * (choices$lagRelevantPayoff > 0)

# Pooled model: treatment x prior win
winStayReg <- feols(
  switch ~ EVAlgorithm * lagWin | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg)
```

```
## OLS estimation, Dep. Var.: switch
## Observations: 1,615
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                      Estimate Std. Error   t value   Pr(>|t|)
## EVAlgorithm          0.104670   0.049740  2.104353 3.6424e-02 *
## lagWin              -0.169470   0.035178 -4.817493 2.6258e-06 ***
## EVAlgorithm:lagWin -0.008762   0.048728 -0.179822 8.5745e-01
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.456921     Adj. R2: 0.052558
##                  Within R2: 0.043757
```

```r
# Appendix table: the EV coefficient is the EV-vs-Pref difference in staying after a
# loss (lagWin = 0); adding the interaction gives the difference after a win
b <- coef(winStayReg); V <- vcov(winStayReg)
gap_win <- b["EVAlgorithm"] + b["EVAlgorithm:lagWin"]
se_win  <- sqrt(V["EVAlgorithm", "EVAlgorithm"] +
                V["EVAlgorithm:lagWin", "EVAlgorithm:lagWin"] +
                2 * V["EVAlgorithm", "EVAlgorithm:lagWin"])
cat("EV vs. Pref after a win:", round(gap_win, 3), " SE", round(se_win, 3),
    " p", round(2 * pnorm(-abs(gap_win / se_win)), 3), "\n")
```

```
## EV vs. Pref after a win: 0.096  SE 0.038  p 0.011
```

Separately by treatment, distinguishing whether the previous round was delegated (`lagDelegated == 1`, i.e. "the algorithm" produced the outcome) from a self-made choice. This separates lose-shift after one's own loss from lose-shift after a delegated (algorithmic) loss.

```r
# EV condition
winStayReg_EV <- feols(
  switch ~ lagDelegated * I(1 - lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVAlgorithm == 1 & Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg_EV)
```

```
## OLS estimation, Dep. Var.: switch
## Observations: 819
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
```

```
## Standard-errors: Clustered (ProlificID)
##                             Estimate Std. Error   t value  Pr(>|t|)
## lagDelegated               -0.044647   0.044771 -0.997214 0.3207185
## I(1 - lagWin)               0.075637   0.050734  1.490836 0.1386957
## lagDelegated:I(1 - lagWin)  0.184727   0.069840  2.645020 0.0092905 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.473051     Adj. R2: 0.03568
##                  Within R2: 0.038964
```

```
# Preference-based condition
winStayReg_Pref <- feols(
  switch ~ lagDelegated * I(1 - lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVAlgorithm == 0 & Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg_Pref)
```

```
## OLS estimation, Dep. Var.: switch
## Observations: 796
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                             Estimate Std. Error   t value Pr(>|t|)
## lagDelegated               -0.024197   0.036355 -0.665574 0.507028
## I(1 - lagWin)               0.108747   0.048066  2.262433 0.025565 *
## lagDelegated:I(1 - lagWin)  0.140381   0.063708  2.203504 0.029570 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.432604     Adj. R2: 0.058073
##                  Within R2: 0.042744
```

```
# Average stay rates by condition, prior delegation, and prior win/loss
choices %>%
  filter(Part == 2, WithinPartDecisionNumber > 1) %>%
  group_by(EVAlgorithm, lagDelegated, lagWin) %>%
  summarise(avg_stay_rate = mean(stay, na.rm = TRUE),
            avg_switch_rate = mean(switch, na.rm = TRUE),
            n = n(), .groups = "drop") %>%
  arrange(EVAlgorithm, lagDelegated, lagWin)
```

```
## # A tibble: 9 x 6
##   EVAlgorithm lagDelegated lagWin avg_stay_rate avg_switch_rate     n
##         <int>        <int>  <dbl>         <dbl>           <dbl> <int>
## 1           0            0      0         0.677           0.323   155
## 2           0            0      1         0.769           0.231   255
## 3           0            1      0         0.547           0.453   139
## 4           0            1      1         0.785           0.215   247
## 5           0           NA     NA       NaN             NaN         2
## 6           1            0      0         0.583           0.417   144
## 7           1            0      1         0.668           0.332   262
## 8           1            1      0         0.444           0.556   169
## 9           1            1      1         0.697           0.303   244
```

7 Exploratory mechanisms

7.1 Coefficient model (interaction with performance expectations)

LPM of individual delegation decisions interacting treatment with participants' algorithmic performance expectations, controlling for round, per-choice Part-1 confidence, revealed Part-1 risk preferences, and task fixed effects.

```
delRegWConf <- feols(
  Delegated ~ EVAlgorithm * (WithinPartDecisionNumber + ConfidenceP1 +
    ProbOfAbsurdComputerChoices + ProbOfBadComputerChoices +
    EVChoices + truePChoices) | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegWConf)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,848
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                          Estimate Std. Error   t value
## EVAlgorithm                              0.856469   0.891957  0.960213
## WithinPartDecisionNumber                -0.009920   0.005711 -1.737089
## ConfidenceP1                            -0.025275   0.012620 -2.002676
## ProbOfAbsurdComputerChoices              0.009864   0.018425  0.535348
## ProbOfBadComputerChoices                -0.060048   0.016324 -3.678547
## EVChoices                                0.067083   0.058904  1.138847
## truePChoices                             0.058960   0.054107  1.089695
## EVAlgorithm:WithinPartDecisionNumber    -0.012262   0.008341 -1.470058
## EVAlgorithm:ConfidenceP1                 0.043682   0.015740  2.775229
## EVAlgorithm:ProbOfAbsurdComputerChoices -0.072908   0.023059 -3.161780
## EVAlgorithm:ProbOfBadComputerChoices     0.036186   0.020603  1.756318
## EVAlgorithm:EVChoices                   -0.089742   0.077330 -1.160496
## EVAlgorithm:truePChoices                -0.095940   0.071561 -1.340675
##                                          Pr(>|t|)
## EVAlgorithm                             0.3379475
## WithinPartDecisionNumber                0.0836987 .
## ConfidenceP1                            0.0463759 *
## ProbOfAbsurdComputerChoices             0.5929221
## ProbOfBadComputerChoices                0.0002916 ***
## EVChoices                               0.2559415
## truePChoices                            0.2769783
## EVAlgorithm:WithinPartDecisionNumber    0.1429009
## EVAlgorithm:ConfidenceP1                0.0059660 **
## EVAlgorithm:ProbOfAbsurdComputerChoices 0.0017772 **
## EVAlgorithm:ProbOfBadComputerChoices    0.0803535 .
## EVAlgorithm:EVChoices                   0.2470398
## EVAlgorithm:truePChoices                0.1813374
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.474486     Adj. R2: 0.089466
##                  Within R2: 0.089764
```

7.2 Treatment comparison of expected algorithmic errors

Formal comparison of expected non-preferred and completely weird choices across treatments. A LOWER expected number of non-preferred choices under the preference-based algorithm would indicate that participants believe the personalized algorithm is doing its job (matching their preferences).

```
expect_tbl <- subjects %>%
  mutate(Condition = ifelse(EVAlgorithm == 1, "EV", "Preference-based")) %>%
  group_by(Condition) %>%
  summarise(
    n                  = n(),
    mean_nonpreferred  = mean(ProbOfBadComputerChoices, na.rm = TRUE),
    sd_nonpreferred    = sd(ProbOfBadComputerChoices, na.rm = TRUE),
    mean_weird         = mean(ProbOfAbsurdComputerChoices, na.rm = TRUE),
    sd_weird           = sd(ProbOfAbsurdComputerChoices, na.rm = TRUE),
    .groups = "drop"
  )
expect_tbl
```

```
## # A tibble: 2 x 6
##   Condition            n mean_nonpreferred sd_nonpreferred mean_weird sd_weird
##   <chr>            <int>             <dbl>           <dbl>      <dbl>    <dbl>
## 1 EV                 118              5               2.32       1.78     1.74
## 2 Preference-based   115              3.78            2.16       1.57     1.62
```

```
# Expected NON-PREFERRED choices: EV vs. Preference-based
t.test(ProbOfBadComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Welch Two Sample t-test
##
## data:  ProbOfBadComputerChoices by EVAlgorithm
## t = -4.1504, df = 230.54, p-value = 4.671e-05
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -1.7953130 -0.6394696
## sample estimates:
## mean in group 0 mean in group 1
##        3.782609        5.000000
```

```
wilcox.test(ProbOfBadComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  ProbOfBadComputerChoices by EVAlgorithm
## W = 4805, p-value = 9.898e-05
## alternative hypothesis: true location shift is not equal to 0
```

```
# Expected COMPLETELY WEIRD choices: EV vs. Preference-based
t.test(ProbOfAbsurdComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Welch Two Sample t-test
##
## data:  ProbOfAbsurdComputerChoices by EVAlgorithm
```

```
## t = -0.93521, df = 230.48, p-value = 0.3507
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -0.6392193  0.2277233
## sample estimates:
## mean in group 0 mean in group 1
##        1.573913        1.779661
```

```
wilcox.test(ProbOfAbsurdComputerChoices ~ EVAlgorithm, data = subjects)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  ProbOfAbsurdComputerChoices by EVAlgorithm
## W = 6328, p-value = 0.361
## alternative hypothesis: true location shift is not equal to 0
```

```
# Within the preference-based condition: do never-delegators expect the
# personalized algorithm to make MORE non-preferred choices than delegators?
# (If not, categorical non-delegation is not driven by distrust in the
#  algorithm's ability to capture one's preferences.)
analysis_data %>%
  filter(EVAlgorithm == 0) %>%
  group_by(NonDelegator) %>%
  summarise(
    n                 = n(),
    mean_nonpreferred = mean(ProbOfBadComputerChoices, na.rm = TRUE),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 3
##   NonDelegator     n mean_nonpreferred
##          <dbl> <int>             <dbl>
## 1            0    97              3.77
## 2            1    18              3.83
```

```
t.test(ProbOfBadComputerChoices ~ NonDelegator,
       data = subset(analysis_data, EVAlgorithm == 0))
```

```
##
##  Welch Two Sample t-test
##
## data:  ProbOfBadComputerChoices by NonDelegator
## t = -0.11119, df = 24.369, p-value = 0.9124
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -1.175546  1.055272
## sample estimates:
## mean in group 0 mean in group 1
##        3.773196        3.833333
```

8 Post-task reasons

8.1 Figure 4 — Reasons by treatment and delegation type

Reason-selection frequencies for the hypothetical additional-task choice, by treatment (EV vs. preference-based) and delegation type (delegators vs. never-delegators).

```
subjects2 <- subjects %>% left_join(NeverDelegate, by = "ProlificID")

subjects_long <- subjects2 %>%
  pivot_longer(cols = DontTrust:Other, names_to = "item", values_to = "clicked")

# Group sizes for legend labels
n_labels <- subjects_long %>%
  group_by(sumDeleg0, EVAlgorithm) %>%
  summarise(n = n_distinct(ProlificID), .groups = "drop") %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE  & EVAlgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE  & EVAlgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 1 ~ "EV--Delegators"
    ),
    group_label = paste0(group, " (n=", n, ")")
  )

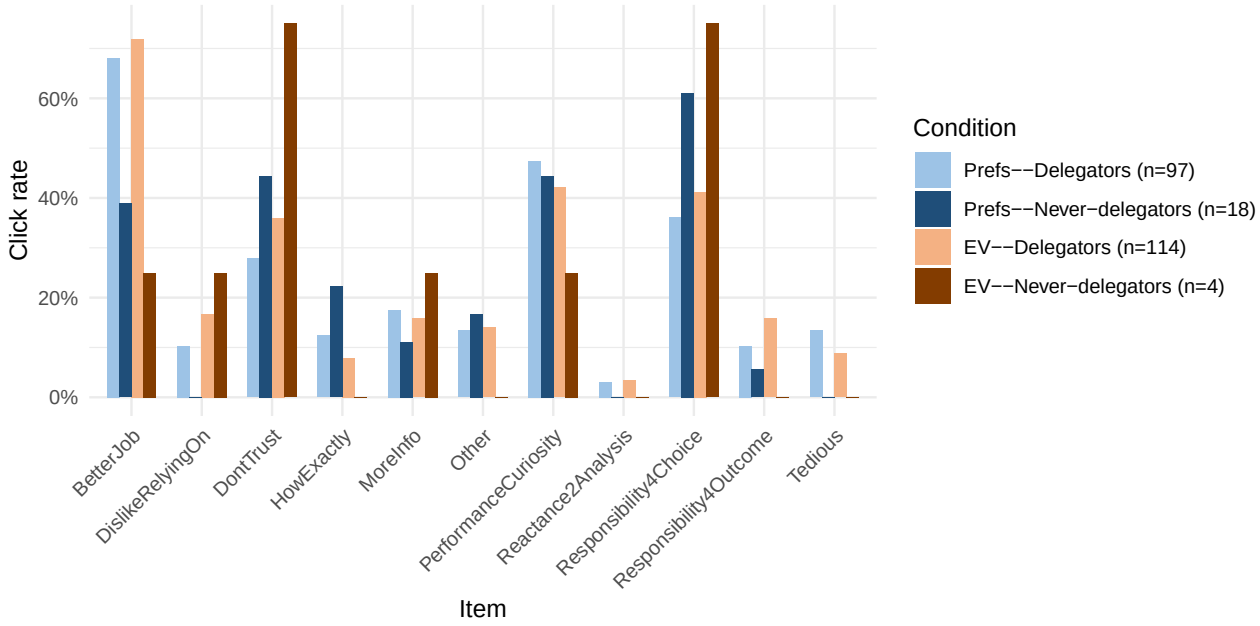
group_colors <- c(
  "Prefs--Delegators"       = "#9dc3e6",
  "Prefs--Never-delegators" = "#1f4e79",
  "EV--Delegators"          = "#f4b183",
  "EV--Never-delegators"    = "#833c00"
)
label_map <- setNames(n_labels$group_label, n_labels$group)

click_rates_both <- subjects_long %>%
  group_by(sumDeleg0, EVAlgorithm, item) %>%
  summarise(rate = mean(clicked, na.rm = TRUE), .groups = "drop") %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE  & EVAlgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE  & EVAlgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 1 ~ "EV--Delegators"
    ),
    group = factor(group, levels = names(group_colors))
  )

fig4 <- ggplot(click_rates_both, aes(x = item, y = rate, fill = group)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.7) +
  scale_fill_manual(values = group_colors, labels = label_map,
                    breaks = names(group_colors), name = "Condition") +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(x = "Item", y = "Click rate") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
```

```
        legend.position = "right")

fig4
```




```
ggsave("Figure4_ReasonsByTrmtAndNeverDelegator2.pdf", fig4, width = 8, height = 4, dpi =
↪  300)
```

8.1.1 Side note: participants who expect few mistakes by the algorithm

The paper notes that restricting the sample to participants who expect at most two non-preferred choices out of ten (about 21% of the sample) leaves the treatment differences at least as large. The subgroups are small, and the restriction conditions on a post-task measure, so we regard this only as a robustness remark. Paper (at most 2): round-1 delegation 100% (EV, n = 15) vs. 72.7% (Pref, n = 33), one-sided Boschloo $p = 0.012$; never-delegation 0 of 15 vs. 6 of 33 ($p = 0.053$). With a cut-off of 3: 83.3% vs. 65.4% ($p = 0.064$) and never-delegation $p = 0.039$.

```
few_mistakes <- subjects %>%
  inner_join(choices %>% filter(Part == 2) %>% group_by(ProlificID) %>%
               summarise(k = sum(Delegated == 1), n = sum(Delegated >= 0),
                         d1 = Delegated[WithinPartDecisionNumber == 1][1], .groups =
                         ↪  "drop"),
             by = "ProlificID")

# one-sided Boschloo on arms (row 1 = EV, row 2 = Pref); "greater" = EV higher
boschloo_ev <- function(x, ev, alternative) {
  tab <- rbind(c(sum(x[ev == 1] == 1), sum(x[ev == 1] == 0)),
               c(sum(x[ev == 0] == 1), sum(x[ev == 0] == 0)))
  exact.test(tab, method = "Boschloo", alternative = alternative, to.plot =
  ↪  FALSE)$p.value
}
for (cutoff in c(2, 3)) {
  d <- filter(few_mistakes, ProbOfBadComputerChoices <= cutoff, !is.na(d1))
  cat(sprintf("\n--- At most %d expected non-preferred choices: n = %d (%.1f%% of %d)\n",
              cutoff, nrow(d), 100 * nrow(d) / nrow(few_mistakes), nrow(few_mistakes)))
```

```
  print(d %>% group_by(EVAlgorithm) %>%
          summarise(n = n(), round1 = round(100 * mean(d1), 1), never = sum(k == 0),
                    neverPct = round(100 * mean(k == 0), 1),
                    avgDelegation = round(100 * mean(k / n), 1), .groups = "drop"))
  cat("Round-1 delegation, EV > Pref (one-sided Boschloo p):",
      round(boschloo_ev(d$d1, d$EVAlgorithm, "greater"), 3), "\n")
  cat("Never-delegation, Pref > EV (one-sided Boschloo p):",
      round(boschloo_ev(1 * (d$k == 0), d$EVAlgorithm, "less"), 3), "\n")
}
```

```
##
## --- At most 2 expected non-preferred choices: n = 48 (20.6% of 233)
## # A tibble: 2 x 6
##   EVAlgorithm     n round1 never neverPct avgDelegation
##         <int> <int>  <dbl> <int>    <dbl>         <dbl>
## 1           0    33   72.7     6     18.2          16.6
## 2           1    15  100       0      0            43.6
## Round-1 delegation, EV > Pref (one-sided Boschloo p): 0.012
## Never-delegation, Pref > EV (one-sided Boschloo p): 0.053
##
## --- At most 3 expected non-preferred choices: n = 76 (32.6% of 233)
## # A tibble: 2 x 6
##   EVAlgorithm     n round1 never neverPct avgDelegation
##         <int> <int>  <dbl> <int>    <dbl>         <dbl>
## 1           0    52   65.4     7     13.5           9.3
## 2           1    24   83.3     0      0            24
## Round-1 delegation, EV > Pref (one-sided Boschloo p): 0.064
## Never-delegation, Pref > EV (one-sided Boschloo p): 0.039
```

8.2 Aggregate reason patterns

In-text figures for performance, distrust, and responsibility motives by delegation type (delegators vs. never-delegators).

```
subjects_long %>%
  group_by(sumDeleg0, item) %>%
  summarise(rate = mean(clicked, na.rm = TRUE), .groups = "drop") %>%
  pivot_wider(names_from = sumDeleg0, values_from = rate) %>%
  rename(Delegators = `FALSE`, NeverDelegators = `TRUE`)
```

```
## # A tibble: 11 x 3
##    item                   Delegators NeverDelegators
##    <chr>                       <dbl>           <dbl>
##  1 BetterJob                  0.701           0.364
##  2 DislikeRelyingOn           0.137           0.0455
##  3 DontTrust                  0.322           0.5
##  4 HowExactly                 0.0995          0.182
##  5 MoreInfo                   0.166           0.136
##  6 Other                      0.137           0.136
##  7 PerformanceCuriosity       0.445           0.409
##  8 Reactance2Analysis         0.0332          0
##  9 Responsibility4Choice      0.389           0.636
## 10 Responsibility4Outcome     0.133           0.0455
## 11 Tedious                    0.109           0
```

8.3 The responsibility items are worded according to the hypothetical choice

The reason items refer to the hypothetical choice (`DelegateHyp10`: 1 = delegate, 0 = decide oneself) and their wording depends on it. For example, `Responsibility4Choice` reads "I wanted to be responsible for choosing the option" for participants who chose to decide themselves and "I didn't want to be responsible for choosing the option" for participants who chose to delegate. The pooled comparison of never-delegators with the rest therefore largely contrasts reasons for deciding oneself with reasons for delegating. This section documents the relationship between the hypothetical choice and actual behavior and reports the endorsement rates within each hypothetical choice. Paper: 91% of never-delegators decide themselves, 95% of always-delegators delegate, Spearman correlation with the Part-2 delegation share 0.68; endorsement of `Responsibility4Choice` is 66.4% among participants who decide themselves and 16.2% among those who delegate; never-delegators who decide themselves do not differ from other participants who decide themselves (70.0% vs. 65.6%; Boschloo $p = 0.80$); about one in four delegators endorses either responsibility item.

```
resp <- subjects2 %>%
  left_join(choices %>% filter(Part == 2) %>% group_by(ProlificID) %>%
              summarise(nValid = sum(Delegated >= 0),
                        delegShare = mean(Delegated[Delegated >= 0]), .groups = "drop"),
            by = "ProlificID") %>%
  mutate(Type = ifelse(sumDelegations == 0, "Never",
                ifelse(sumDelegations == nValid, "Always", "Sometimes")),
         Hypothetical = ifelse(DelegateHyp10 == 1, "Delegate", "Decide myself"))

cat("Hypothetical choice by actual delegation type\n")
```

```
## Hypothetical choice by actual delegation type
```

```
table(Hypothetical = resp$Hypothetical, Type = resp$Type)
```

```
##                Type
## Hypothetical    Always Never Sometimes
##   Decide myself      2    20        94
##   Delegate          38     2        77
```

```
cat("\nColumn percentages\n")
```

```
##
## Column percentages
```

```
round(100 * prop.table(table(resp$Hypothetical, resp$Type), 2), 1)
```

```
##
##                 Always Never Sometimes
##   Decide myself    5.0  90.9      55.0
##   Delegate        95.0   9.1      45.0
```

```
cat("\nMean Part-2 delegation share by hypothetical choice\n")
```

```
##
## Mean Part-2 delegation share by hypothetical choice
```

```
resp %>% group_by(Hypothetical) %>%
  summarise(n = n(), meanShare = round(mean(delegShare), 3), .groups = "drop")
```

```
## # A tibble: 2 x 3
##   Hypothetical      n meanShare
##   <chr>         <int>     <dbl>
```

```
## 1 Decide myself   116     0.268
## 2 Delegate        117     0.72
```

```
cat("Spearman correlation, delegation share vs. hypothetical choice:",
    round(cor(resp$delegShare, resp$DelegateHyp10, method = "spearman"), 3), "\n")
```

```
## Spearman correlation, delegation share vs. hypothetical choice: 0.679
```

```
sometimes <- filter(resp, Type == "Sometimes")
cat("  among sometimes-delegators only:",
    round(cor(sometimes$delegShare, sometimes$DelegateHyp10, method = "spearman"), 3),
    ↪  "\n")
```

```
##   among sometimes-delegators only: 0.607
```

```
cat("\nResponsibility items by hypothetical choice (percent endorsing)\n")
```

```
##
## Responsibility items by hypothetical choice (percent endorsing)
```

```
resp %>%
  group_by(Hypothetical) %>%
  summarise(n = n(),
            Responsibility4Choice  = round(100 * mean(Responsibility4Choice == 1, na.rm =
            ↪  TRUE), 1),
            Responsibility4Outcome = round(100 * mean(Responsibility4Outcome == 1, na.rm
            ↪  = TRUE), 1),
            .groups = "drop")
```

```
## # A tibble: 2 x 4
##   Hypothetical      n Responsibility4Choice Responsibility4Outcome
##   <chr>         <int>                 <dbl>                  <dbl>
## 1 Decide myself   116                  66.4                    7.8
## 2 Delegate        117                  16.2                   17.1
```

```
cat("\nAmong participants who chose to delegate: either responsibility item\n")
```

```
##
## Among participants who chose to delegate: either responsibility item
```

```
deleg_hyp <- filter(resp, DelegateHyp10 == 1)
cat("  n =", nrow(deleg_hyp), " share =",
    round(100 * mean(deleg_hyp$Responsibility4Choice == 1 |
                     deleg_hyp$Responsibility4Outcome == 1, na.rm = TRUE), 1), "%\n")
```

```
##   n = 117  share = 25.6 %
```

```
cat("\nResponsibility4Choice by actual type and hypothetical choice\n")
```

```
##
## Responsibility4Choice by actual type and hypothetical choice
```

```
resp %>%
  group_by(Type, Hypothetical) %>%
  summarise(n = n(), pct = round(100 * mean(Responsibility4Choice == 1, na.rm = TRUE),
  ↪  1),
            .groups = "drop")
```

```
## # A tibble: 6 x 4
##   Type      Hypothetical      n   pct
##   <chr>     <chr>         <int> <dbl>
## 1 Always    Decide myself     2  50
## 2 Always    Delegate         38  13.2
## 3 Never     Decide myself    20  70
## 4 Never     Delegate          2   0
## 5 Sometimes Decide myself    94  66
## 6 Sometimes Delegate         77  18.2
```

```
cat("\nWithin participants who decide themselves: never-delegators vs. others
↪  (Boschloo)\n")
```

```
##
## Within participants who decide themselves: never-delegators vs. others (Boschloo)
```

```
own <- filter(resp, DelegateHyp10 == 0)
tab_own <- table(factor(1 * (own$Type == "Never"), levels = c(1, 0)),
                 factor(own$Responsibility4Choice, levels = c(1, 0)))
print(tab_own)
```

```
##
##      1  0
##   1 14  6
##   0 63 33
```

```
cat("  two-sided Boschloo p =",
    round(exact.test(tab_own, method = "Boschloo", to.plot = FALSE)$p.value, 3), "\n")
```

```
##   two-sided Boschloo p = 0.799
```

```
cat("\nTreatment differences within each hypothetical choice (Responsibility4Choice)\n")
```

```
##
## Treatment differences within each hypothetical choice (Responsibility4Choice)
```

```
for (h in c("Delegate", "Decide myself")) {
  d <- filter(resp, Hypothetical == h)
  tb <- table(factor(d$EVAlgorithm, levels = c(0, 1)),
              factor(d$Responsibility4Choice, levels = c(1, 0)))
  cat(" ", h, ": Pref", round(100 * mean(d$Responsibility4Choice[d$EVAlgorithm == 0] ==
   ↪  1), 1),
      "% vs. EV", round(100 * mean(d$Responsibility4Choice[d$EVAlgorithm == 1] == 1), 1),
      "%; Boschloo p =", round(exact.test(tb, method = "Boschloo", to.plot =
       ↪  FALSE)$p.value, 3), "\n")
}
```

```
##   Delegate : Pref 16.1 % vs. EV 16.4 %; Boschloo p = 1
##   Decide myself : Pref 62.7 % vs. EV 70.2 %; Boschloo p = 0.404
```

8.4 Reasons among delegators, by treatment

The paper notes that, among participants who delegated at least once, the selected reasons differ little by treatment (by at most 8 percentage points; smallest two-sided Boschloo $p = 0.214$). The four EV never-delegators are too few to compare.

```r
delegator_reasons <- subjects2 %>% filter(sumDeleg0 == FALSE)
items_closed <- c("DontTrust", "BetterJob", "HowExactly", "MoreInfo",
↪  "Responsibility4Choice",
                  "Responsibility4Outcome", "DislikeRelyingOn", "Reactance2Analysis",
                  "PerformanceCuriosity", "Tedious")
res <- t(sapply(items_closed, function(v) {
  tab <- table(factor(delegator_reasons$EVAlgorithm, levels = c(0, 1)),
               factor(delegator_reasons[[v]], levels = c(1, 0)))
  c(Pref = 100 * mean(delegator_reasons[[v]][delegator_reasons$EVAlgorithm == 0] == 1,
  ↪  na.rm = TRUE),
    EV   = 100 * mean(delegator_reasons[[v]][delegator_reasons$EVAlgorithm == 1] == 1,
    ↪  na.rm = TRUE),
    p    = exact.test(tab, method = "Boschloo", to.plot = FALSE)$p.value)
}))
round(res, 3)
```

```
##                           Pref     EV     p
## DontTrust               27.835 35.965 0.217
## BetterJob               68.041 71.930 0.523
## HowExactly              12.371  7.895 0.334
## MoreInfo                17.526 15.789 0.830
## Responsibility4Choice   36.082 41.228 0.463
## Responsibility4Outcome  10.309 15.789 0.287
## DislikeRelyingOn        10.309 16.667 0.214
## Reactance2Analysis       3.093  3.509 1.000
## PerformanceCuriosity    47.423 42.105 0.476
## Tedious                 13.402  8.772 0.349
```

```r
cat("Largest difference (percentage points):", round(max(abs(res[, "Pref"] - res[,
↪  "EV"])), 1), "\n")
```

```
## Largest difference (percentage points): 8.1
```

8.5 LPM: never-delegation on reasons

```r
reason_vars <- c("DontTrust", "BetterJob", "HowExactly", "MoreInfo",
                 "Responsibility4Choice", "Responsibility4Outcome",
                 "DislikeRelyingOn", "Reactance2Analysis",
                 "PerformanceCuriosity", "Tedious", "Other")

# Bivariate LPMs (one per reason, HC2 SEs)
bivariate_tbl <- do.call(rbind, lapply(reason_vars, function(rv) {
  m  <- lm(as.formula(paste("as.numeric(sumDeleg0) ~", rv)), data = subjects2)
  ct <- coeftest(m, vcov = vcovHC(m, type = "HC2"))
  data.frame(reason = rv,
             coef = round(ct[2, 1], 3), se = round(ct[2, 2], 3),
             t = round(ct[2, 3], 3), p = round(ct[2, 4], 3))
}))
print(bivariate_tbl, row.names = FALSE)
```

```
##                 reason   coef    se      t     p
##              DontTrust  0.068 0.044  1.528 0.128
##              BetterJob -0.131 0.048 -2.739 0.007
##             HowExactly  0.073 0.077  0.950 0.343
```

```
##                MoreInfo -0.018 0.049 -0.376 0.707
##   Responsibility4Choice  0.087 0.041  2.111 0.036
##  Responsibility4Outcome -0.068 0.041 -1.688 0.093
##        DislikeRelyingOn -0.070 0.040 -1.769 0.078
##      Reactance2Analysis -0.097 0.020 -4.926 0.000
##    PerformanceCuriosity -0.013 0.038 -0.328 0.743
##                 Tedious -0.105 0.021 -4.945 0.000
##                   Other -0.001 0.056 -0.014 0.989
```

```r
# Joint LPM: full sample + preference-treatment only
f_joint  <- as.formula(paste("as.numeric(sumDeleg0) ~",
                             paste(reason_vars, collapse = " + ")))
lpm_all  <- lm(f_joint, data = subjects2)
lpm_pref <- lm(f_joint, data = subjects2[subjects2$EVAlgorithm == 0, ])

vcov_all  <- vcovHC(lpm_all,  type = "HC2")
vcov_pref <- vcovHC(lpm_pref, type = "HC2")

screenreg(
  list(lpm_all, lpm_pref),
  override.se = list(sqrt(diag(vcov_all)), sqrt(diag(vcov_pref))),
  override.pvalues = list(
    2 * pt(abs(coef(lpm_all)  / sqrt(diag(vcov_all))),
           df = lpm_all$df.residual,  lower.tail = FALSE),
    2 * pt(abs(coef(lpm_pref) / sqrt(diag(vcov_pref))),
           df = lpm_pref$df.residual, lower.tail = FALSE)
  ),
  custom.model.names = c("Full sample", "Pref. treatment"),
  caption = "LPM: never-delegation ~ reasons (HC2 SEs)"
)
```

```
##
## ====================================================
##                          Full sample  Pref. treatment
## ----------------------------------------------------
## (Intercept)               0.14 **      0.20 *
##                          (0.05)       (0.09)
## DontTrust                 0.08         0.15
##                          (0.04)       (0.09)
## BetterJob                -0.11 *      -0.14
##                          (0.05)       (0.08)
## HowExactly                0.04         0.05
##                          (0.08)       (0.10)
## MoreInfo                 -0.04        -0.15
##                          (0.06)       (0.11)
## Responsibility4Choice     0.06         0.11
##                          (0.04)       (0.07)
## Responsibility4Outcome   -0.04        -0.00
##                          (0.04)       (0.10)
## DislikeRelyingOn         -0.09 *      -0.25 **
##                          (0.04)       (0.08)
## Reactance2Analysis       -0.09        -0.12
##                          (0.04)       (0.12)
## PerformanceCuriosity      0.02         0.03
```

```
##                           (0.04)       (0.07)
## Tedious                  -0.07 **     -0.16 **
##                           (0.03)       (0.06)
## Other                    -0.02        -0.01
##                           (0.06)       (0.12)
## ---------------------------------------------------
## R^2                       0.10         0.16
## Adj. R^2                  0.05         0.06
## Num. obs.                233          115
## ===================================================
## *** p < 0.001; ** p < 0.01; * p < 0.05
```

9 Exclusion-criterion breakdown (Method section)

Reconstructs the exact breakdown of the 7 excluded participants by criterion, as reported in the Method section (paper: 1 bot/slider failure, 4 careful-participation failures [`Reliable < 6`], 3 disclosed-AI-usage cases, 0 comprehension-only failures, 0 incomplete-data cases, 0 log-time exclusions; final $N = 233$, with 1 participant overlapping careful-participation-failure and AI-disclosure). This chunk re-reads the raw files independently of the exclusions already applied above (using `_raw` suffixes), so it is self-contained and can be run at any point in the script.

```
choices_raw      <- read.delim("260206_1528_537_lotteryChoices.xls")
prolificData_raw <- read.csv("prolific_demographic_export_approved.csv")

cat("N recruited (Prolific export):", nrow(prolificData_raw), "\n")
```

```
## N recruited (Prolific export): 240
```

```
choices_raw <- subset(choices_raw, ProlificID %in% prolificData_raw$Participant.id)

part2_raw <- subset(choices_raw, Part == 2)
partvars <- part2_raw[!duplicated(part2_raw$ProlificID),
                      c("ProlificID", "BotCheck1attempt", "Part1CQsPassedOnAttempt",
                        "Part2CQsPassedOnAttempt", "Reliable", "AIUsageInStudy")]

# Incomplete-data flag: participant has zero valid (Delegated >= 0) Part-2 rows
valid_counts <- aggregate(Delegated ~ ProlificID, data = part2_raw,
                          FUN = function(x) sum(x >= 0))
names(valid_counts)[2] <- "nValid"
partvars <- merge(partvars, valid_counts, by = "ProlificID", all.x = TRUE)
partvars$incomplete <- is.na(partvars$nValid) | partvars$nValid == 0

# Log-completion-time "too fast" rule
prolificData_raw$logCompleTime <- log(prolificData_raw$Time.taken)
prolificData_raw$tooFast <- 1 * (
  prolificData_raw$logCompleTime <
    mean(log(prolificData_raw$Time.taken)) - 2 * sd(log(prolificData_raw$Time.taken))
)
partvars <- merge(partvars, prolificData_raw[, c("Participant.id", "tooFast")],
                  by.x = "ProlificID", by.y = "Participant.id", all.x = TRUE)

partvars$failBot      <- partvars$BotCheck1attempt != 1
partvars$failComp     <- partvars$Part1CQsPassedOnAttempt > 2 |
                         partvars$Part2CQsPassedOnAttempt > 2
partvars$failCareful  <- partvars$Reliable < 6
partvars$disclosedAI  <- partvars$AIUsageInStudy != 0

excluded <- subset(
  partvars,
  failBot | failComp | failCareful | disclosedAI | incomplete | tooFast == 1
)

cat("\nTotal excluded:", nrow(excluded), "\n")
```

```
##
## Total excluded: 7
```

```r
cat("Fail bot/slider check:", sum(excluded$failBot), "\n")
```

```
## Fail bot/slider check: 1
```

```r
cat("Fail comprehension check only:", sum(excluded$failComp), "\n")
```

```
## Fail comprehension check only: 0
```

```r
cat("Fail careful-participation check (Reliable < 6):", sum(excluded$failCareful), "\n")
```

```
## Fail careful-participation check (Reliable < 6): 4
```

```r
cat("Disclosed AI usage:", sum(excluded$disclosedAI), "\n")
```

```
## Disclosed AI usage: 3
```

```r
cat("Incomplete data:", sum(excluded$incomplete), "\n")
```

```
## Incomplete data: 0
```

```r
cat("Too fast (log-time rule):", sum(excluded$tooFast == 1, na.rm = TRUE), "\n")
```

```
## Too fast (log-time rule): 0
```

```r
cat("Overlap (careful-participation AND disclosed AI):",
    sum(excluded$failCareful & excluded$disclosedAI), "\n")
```

```
## Overlap (careful-participation AND disclosed AI): 1
```

```r
cat("Final analysis N:", nrow(partvars) - nrow(excluded), "\n")
```

```
## Final analysis N: 233
```

10 Power analysis (pilot-based, Appendix C of the paper)

Reconstructs the ex-ante power analysis reported in Appendix C (summarized in the Limitations): using only the pilot's effect sizes (not the present study's own results, to avoid the retrospective-power fallacy; Hoenig & Heisey, 2001), we compute the minimum detectable effect (MDE) at 80% power for $n = 100$–120/arm, and the power to re-detect an effect of the pilot's observed magnitude. Pilot data: `pilot_data_complete_iw.csv` (an earlier, non-preregistered study reported in Suraci's PhD thesis).

```
pilot <- read.csv("pilot_data_complete_iw.csv")
pilot <- subset(pilot, returned == 1)
cat("Pilot N (returned == 1):", nrow(pilot), "\n")
```

```
## Pilot N (returned == 1): 377
```

```
# EV treatments = 1,2; preference-based (RDU) treatments = 3,4
pilot$EValgo <- 1 * (pilot$treatment %in% c(1, 2))
cat("\nPilot N by condition (0 = Pref, 1 = EV):\n")
```

```
##
## Pilot N by condition (0 = Pref, 1 = EV):
```

```
print(table(pilot$EValgo))
```

```
##
##   0   1
## 190 187
```

```
# First-contact delegation (randomized-order "1st" decision)
cat("\nFirst-contact delegation rate by condition (paper: 45.8% Pref vs. 67.4% EV):\n")
```

```
##
## First-contact delegation rate by condition (paper: 45.8% Pref vs. 67.4% EV):
```

```
print(aggregate(decision1st ~ EValgo, data = pilot, mean))
```

```
##   EValgo decision1st
## 1      0   0.4578947
## 2      1   0.6737968
```

```
# Average delegation across all 8 (randomized-order) decisions
decisioncols <- c("decision1st", "decision2nd", "decision3rd", "decision4th",
                  "decision5th", "decision6th", "decision7th", "decision8th")
pilot$avgDeleg <- rowMeans(pilot[, decisioncols], na.rm = TRUE)
cat("\nAverage delegation rate by condition (paper: 35.3% Pref vs. 54.5% EV):\n")
```

```
##
## Average delegation rate by condition (paper: 35.3% Pref vs. 54.5% EV):
```

```
print(aggregate(avgDeleg ~ EValgo, data = pilot, mean))
```

```
##   EValgo  avgDeleg
## 1      0 0.3532895
## 2      1 0.5447861
```

10.1 Pilot: categorical non-delegation (footnote and Discussion)

The footnote in "Categorical non-delegation" and the Discussion report never-delegation in the earlier study (paper: 32/189 in preference-based vs. 17/187 in EV treatments, one-sided Boschloo $p = 0.014$). **Check:** with

the pilot data as stored here, the preference-based arm has 190 participants (as in the first-contact comparison above), so the denominator should be 190 unless one participant was dropped in the thesis analysis.

```
pilot$nDeleg <- rowSums(pilot[, decisioncols])
pilotNever <- table(
  Treatment = factor(pilot$EValgo, levels = c(0, 1), labels = c("Pref", "EV")),
  Never     = factor(pilot$nDeleg == 0, levels = c(TRUE, FALSE),
                     labels = c("Never", "NotNever"))
)
pilotNever
```

```
##           Never
## Treatment Never NotNever
##      Pref    32      158
##      EV      17      170
```

```
# one-sided Boschloo (Pref > EV), treatment arms as the binomial groups
exact.test(pilotNever, method = "Boschloo", alternative = "greater",
           to.plot = FALSE)$p.value
```

```
## [1] 0.01541084
```

10.2 Minimum detectable effect (MDE) at 80% power

A pure design-stage quantity: depends only on n, α, and a baseline rate, not on anything observed in the present study (paper: ~16-17pp for first contact, ~10-11pp for average delegation).

```
p_pref_firstcontact <- aggregate(decision1st ~ EValgo, data = pilot, mean)[1, 2]

for (n in c(100, 120)) {
  pt <- power.prop.test(n = n, p1 = p_pref_firstcontact, p2 = NULL,
                        power = 0.80, sig.level = 0.05, alternative = "one.sided")
  cat("n =", n, " MDE (first-contact-type proportion) =",
      round(pt$p2 - p_pref_firstcontact, 3), "\n")
}
```

```
## n = 100  MDE (first-contact-type proportion) = 0.174
## n = 120  MDE (first-contact-type proportion) = 0.159
```

10.3 MDE and power for the average-delegation comparison

Approximated with a one-sided two-sample t-test using the pilot's pooled within-condition SD of participant-average delegation (paper: MDE ~10-11pp; 99.7-99.9% power to re-detect the pilot's ~19pp gap).

```
sd_pilot    <- sqrt(mean(tapply(pilot$avgDeleg, pilot$EValgo, var)))
delta_pilot <- diff(tapply(pilot$avgDeleg, pilot$EValgo, mean))
cat("Pilot pooled SD:", round(sd_pilot, 3), " pilot gap:", round(delta_pilot, 3), "\n")
```

```
## Pilot pooled SD: 0.308  pilot gap: 0.191
```

```
for (n in c(100, 120)) {
  m <- power.t.test(n = n, sd = sd_pilot, power = 0.80, sig.level = 0.05,
                    alternative = "one.sided")
  r <- power.t.test(n = n, delta = delta_pilot, sd = sd_pilot, sig.level = 0.05,
                    alternative = "one.sided")
  cat("n =", n, " MDE =", round(m$delta, 3), " power to re-detect pilot gap =",
```

```
      round(r$power, 3), "\n")
}
```

```
## n = 100  MDE = 0.109  power to re-detect pilot gap = 0.997
## n = 120  MDE = 0.099  power to re-detect pilot gap = 0.999
```

10.4 Power to re-detect the pilot's observed effect

Not a retrospective-power computation on the present study's own effect (which would be circular); this uses only the pilot's pre-existing effect size as input (paper: 93-96% under the normal approximation, 91-95% by exact-test simulation).

```
p_ev_firstcontact <- aggregate(decision1st ~ EValgo, data = pilot, mean)[2, 2]

for (n in c(100, 120)) {
  pt <- power.prop.test(n = n, p1 = p_pref_firstcontact, p2 = p_ev_firstcontact,
                        sig.level = 0.05, alternative = "one.sided")
  cat("n =", n, " power to re-detect pilot's first-contact gap =",
      round(pt$power, 3), "\n")
}
```

```
## n = 100  power to re-detect pilot's first-contact gap = 0.929
## n = 120  power to re-detect pilot's first-contact gap = 0.962
```

```
sessionInfo()
```

```
## R version 4.6.1 (2026-06-24)
## Platform: x86_64-pc-linux-gnu
## Running under: TUXEDO OS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0  LAPACK version 3.12.0
##
## locale:
##  [1] LC_CTYPE=de_CH.UTF-8       LC_NUMERIC=C
##  [3] LC_TIME=de_CH.UTF-8        LC_COLLATE=de_CH.UTF-8
##  [5] LC_MONETARY=de_CH.UTF-8    LC_MESSAGES=de_CH.UTF-8
##  [7] LC_PAPER=de_CH.UTF-8       LC_NAME=C
##  [9] LC_ADDRESS=C               LC_TELEPHONE=C
## [11] LC_MEASUREMENT=de_CH.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Europe/Zurich
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] texreg_1.39.5  sandwich_3.1-1 lmtest_0.9-40  zoo_1.8-15     fixest_0.14.1
##  [6] Exact_3.3      scales_1.4.0   ggplot2_4.0.3  tidyr_1.3.2    dplyr_1.2.1
##
## loaded via a namespace (and not attached):
##  [1] utf8_1.2.6          generics_0.1.4      dreamerr_1.5.0
##  [4] lattice_0.23-1      digest_0.6.39       magrittr_2.0.5
```

```
##  [7] evaluate_1.0.5      grid_4.6.1          RColorBrewer_1.1-3
## [10] fastmap_1.2.0       Formula_1.2-5       tinytex_0.59
## [13] httr_1.4.8          purrr_1.2.2         stringmagic_1.2.0
## [16] numDeriv_2016.8-1.1 textshaping_1.0.5   cli_3.6.6
## [19] rlang_1.2.0         withr_3.0.2         yaml_2.3.12
## [22] rootSolve_1.8.2.4   tools_4.6.1         vctrs_0.7.3
## [25] R6_2.6.1            lifecycle_1.0.5     ragg_1.5.2
## [28] pkgconfig_2.0.3     pillar_1.11.1       gtable_0.3.6
## [31] glue_1.8.1          Rcpp_1.1.1-1.1      systemfonts_1.3.2
## [34] xfun_0.57           tibble_3.3.1        tidyselect_1.2.1
## [37] rstudioapi_0.18.0   knitr_1.51          farver_2.1.2
## [40] htmltools_0.5.9     nlme_3.1-171        rmarkdown_2.31
## [43] labeling_0.4.3      compiler_4.6.1      S7_0.2.2
```